# Testing de smart contracts
# Smart contracts testing




Trabajo de Fin de Grado
Curso 2025–2026

Autora
Alicia Grado Guerrero

Directores
Pablo Gordillo Alguacil
Jesús Correas Fernández

Grado en Ingeniería de Computadores
Facultad de Informática
Universidad Complutense de Madrid

# Testing de smart contracts
# Smart contracts testing

**Trabajo de Fin de Grado en Ingeniería de Computadores**


## Autora
**Alicia Grado Guerrero**

## Directores
**Pablo Gordillo Alguacil**
**Jesús Correas Fernández**


**Convocatoria:** *Junio 2026*
**Calificación:** *9,8*


**Grado en Ingeniería de Computadores**
**Facultad de Informática**
**Universidad Complutense de Madrid**


**11 de junio de 2026**

# Agradecimientos

A mis tutores, Pablo y Jesús, muchas gracias por vuestra paciencia y apoyo durante todo el proceso del TFG, y por nuestras reuniones llenas de archivos txt con cosas interesantes.

A mis padres, que siempre me han ayudado en todo lo que he necesitado y a mi hermano, mi modelo a seguir.

A mi amigo y compañero Fari, aunque no hayamos podido presentar el TFG juntos, gracias por todo tu esfuerzo y dedicación.

A mis seres queridos y amigos, gracias por estar ahí siempre y darme ánimos cuando lo he necesitado.

A Andrés y Sánchez, por acompañarnos durante tantos momentos en la cafetería y darnos siempre ánimos.

Gracias a todos.

# Resumen


## Testing de smart contracts

La naturaleza inmutable de la tecnología *blockchain* implica que, una vez desplegado un contrato inteligente, el código no puede ser modificado. Esta característica, sumada al valor económico que gestionan estos programas, hace que cualquier vulnerabilidad sea crítica y potencialmente irreversible. Por ello, la validación del *software* en este entorno no es opcional, sino un requisito de seguridad fundamental para evitar los robos y fallos que frecuentemente afectan al ecosistema.

Este Trabajo de Fin de Grado se centra en el estudio de herramientas de *testing* aplicadas al desarrollo en plataformas *blockchain*. En particular, se estudian las herramientas Foundry y Echidna, que son dos representantes de las herramientas más utilizadas por la comunidad de desarrolladores de *smart contracts*.

Aunque se analizan las pruebas unitarias tradicionales, el núcleo de la investigación profundiza en técnicas de análisis dinámico avanzadas, específicamente el *Mutation-Based Fuzzing*. Utilizando Foundry y Echidna, este trabajo demuestra cómo la generación automatizada de entradas mutadas —guiada por la cobertura del código— es capaz de detectar errores lógicos, vulnerabilidades de reentrada e ineficiencias en el consumo de gas.



## Palabras clave

*Blockchain*, Contrato inteligente, *Mutation-Based Fuzzing*, *Testing*, Foundry, Echidna.

# Abstract


## Smart contracts testing

The immutable nature of blockchain technology implies that once a smart contract is deployed, the code can not be modified. This characteristic, combined with the economic value managed by these programs, makes any vulnerability critical and potentially irreversible. Therefore, software testing in this environment is not optional, but a fundamental security requirement to prevent thefts and failures that frequently affect the ecosystem.

This Bachelor's Thesis focuses on the study of testing tools applied to development on blockchain platforms. In particular, Foundry and Echidna are studied, as they are two representatives of the tools most widely used by the smart contract development community.

Although traditional unit testing is analyzed, the core of the research delves into advanced dynamic analysis techniques, specifically Mutation-Based Fuzzing (MBF). Using Foundry and Echidna, this work demonstrates how automated generation of mutated inputs—guided by code coverage—is capable of detecting logical errors, reentrancy vulnerabilities and inefficiencies in gas consumption.



## Keywords

Blockchain, Smart contract, Mutation-Based Fuzzing, Testing, Foundry, Echidna.

# Índice


**1. Introducción** **1**
- 1.1. Motivación . . . 1
- 1.2. Objetivos . . . 2
- 1.3. Plan de trabajo . . . 2
- 1.4. Estructura de la Memoria . . . 3

**2. Preliminares** **5**
- 2.1. Blockchain . . . 5
- 2.2. Ethereum . . . 6
- 2.3. Testing de software . . . 7
  - 2.3.1. Fuzzing . . . 7
  - 2.3.2. Mutation Testing . . . 8
  - 2.3.3. Mutation-based Fuzzing . . . 8
- 2.4. Testing para contratos inteligentes . . . 9

**3. Herramientas y Análisis de Arquitectura Interna** **11**
- 3.1. Foundry . . . 11
  - 3.1.1. Ejecución de tests . . . 11
- 3.2. Echidna . . . 15
  - 3.2.1. Ejecución de tests . . . 16

**4. Casos de Uso e Implementación** **21**
- 4.1. Evaluación funcional de los contratos . . . 21
  - 4.1.1. Diseño del contrato de prueba . . . 21
  - 4.1.2. Diseño del conjunto de pruebas . . . 23
  - 4.1.3. Evaluación por herramienta . . . 23
- 4.2. Evaluación del consumo de gas . . . 27
  - 4.2.1. Diseño del conjunto de pruebas . . . 28
  - 4.2.2. Análisis del consumo de gas . . . 30
  - 4.2.3. Evaluación por herramienta . . . 33
- 4.3. Evaluación de vulnerabilidades: Reentrada . . . 37
  - 4.3.1. Contexto y concepto de la vulnerabilidad . . . 38

4.3.2. Evaluación con Foundry . . . 40
4.3.3. Evaluación con Echidna . . . 41

**5. Análisis Comparativo de las herramientas 47**
5.1. Foundry . . . 47
5.1.1. Instalación de la herramienta . . . 47
5.1.2. Facilidad de uso . . . 47
5.1.3. Ventajas . . . 48
5.1.4. Desventajas . . . 48
5.2. Echidna . . . 49
5.2.1. Instalación de la herramienta . . . 49
5.2.2. Facilidad de uso . . . 49
5.2.3. Ventajas . . . 50
5.2.4. Desventajas . . . 50
5.3. Comparativa . . . 51

**6. Conclusiones y Trabajo Futuro 53**
6.1. Conclusiones . . . 53
6.2. Líneas de trabajo futuro . . . 54
6.2.1. Pruebas con anvil . . . 54
6.2.2. Estudio de comandos de Echidna . . . 54
6.2.3. Vulnerabilidad de control de acceso . . . 54
6.3. Contribuciones personales . . . 55

**Introduction 57**
6.4. Motivation . . . 57
6.5. Objectives . . . 57
6.6. Work plan . . . 58
6.7. Structure of the report . . . 59

**Conclusions and Future Work 61**
6.8. Conclusions . . . 61
6.9. Future work . . . 61
6.9.1. Testing with anvil . . . 62
6.9.2. Study of Echidna commands . . . 62
6.9.3. Access control vulnerability . . . 62
6.10. Personal Contributions . . . 62

**A. Código pruebas 69**

# Índice de figuras

2.1. Función de ejemplo. . . . . 7

4.1. Código fuente del contrato utilizado para las pruebas funcionales. . . 22
4.2. Código fuente de las pruebas funcionales para Foundry. . . . . 24
4.3. Código fuente de las pruebas funcionales para Echidna. . . . . 25
4.4. Resultado de los *tests* de funcionalidad de Foundry. . . . . 26
4.5. Resultado de los *tests* de funcionalidad de Foundry tras introducir un error en la función `increment`. . . . . 26
4.6. Resultado de los *tests* de funcionalidad de Echidna. . . . . 27
4.7. Resultado de los *tests* de funcionalidad de Echidna tras introducir un error en la función `increment`. . . . . 28
4.8. Código fuente del contrato utilizado para las pruebas de consumo de gas. . . . . 29
4.9. Ejemplo *test* de gas de Echidna. . . . . 30
4.10. Código de las pruebas de consumo de gas (parte 1). . . . . 31
4.11. Código de las pruebas de consumo de gas (parte 2). . . . . 32
4.12. Resultado de los *tests* de gas de Foundry. . . . . 34
4.13. Resultado de los *tests* de gas de Foundry tras reducir los valores de las constantes de gas máximo. . . . . 34
4.14. Resultado del reporte de gas en Foundry para la función `setNumber`. . 35
4.15. Resultado del reporte de gas en Foundry para la función `increment`. . 36
4.16. Resultado del reporte de gas en Foundry para la función `double`. . . . 37
4.17. Resultado del reporte de gas en Foundry para la función `checkcalldata`. 38
4.18. Resultado de los *tests* de gas en Echidna. . . . . 39
4.19. Resultado de los *tests* de gas en Echidna tras reducir las constantes de gas máximo consumido. . . . . 40
4.20. Código fuente del contrato `VulnerableBank.sol`. . . . . 41
4.21. Código fuente del contrato `MaliciousAttacker`. . . . . 42
4.22. Código fuente del entorno de pruebas `ReentrancyTest.sol`. . . . . 43
4.23. Resultado de la evaluación de reentrada y traza de llamadas utilizando Foundry. . . . . 44
4.24. Código fuente de `MaliciousAttacker.sol` para Echidna. . . . . 44
4.25. Código fuente de los *tests* desarrollados para Echidna. . . . . 45

4.26. Contenido `config.yaml`. . . . . 45
4.27. Resultado de la evaluación de Reentrancy y traza de llamadas utilizando Echidna. . . . . 46

# Índice de tablas

5.1. Comparación de herramientas de testing. . . . . 51

Capítulo 1

# Introducción

*"El Bitcoin hará con los bancos lo que el correo electrónico hizo con la industria postal."*
— Rick Flkvinge

*"El mayor riesgo es no asumir ningún riesgo. En un mundo que cambia rápidamente, la única estrategia que garantiza el fracaso es no asumir riesgos."*
— Alistair Cockburn

## 1.1. Motivación

El uso de los *smart contracts* cada vez está más extendido. Durante el cuarto trimestre de 2025 se han desplegado 8,7 millones de contratos inteligentes nuevos sólo en Ethereum [10]. Estos contratos a menudo pueden gestionar grandes cantidades de dinero, por lo que su correcto funcionamiento no es opcional.

Al desplegar un contrato en Ethereum, este se vuelve inmutable, por lo que si hay algún fallo es muy complicado modificarlo. Además, el código compilado del contrato es público, por lo que, si se despliega en la red un contrato con fallos, los atacantes podrían explotarlos ocasionando pérdidas económicas o impidiendo su funcionamiento esperado [21].

No es la primera vez que se ha desplegado un contrato con vulnerabilidades y atacantes han conseguido explotar el fallo haciéndose con grandes cantidades de dinero. Por esta razón probar los contratos antes de usarlos en el mundo real es importante.

Dada la importancia del *testing* de *smart contracts*, resulta de interés analizar distintas herramientas que permitan realizar pruebas sobre contratos inteligentes, analizando su funcionamiento interno y las capacidades de cada herramienta. Este trabajo se centra en dos herramientas: Foundry y Echidna.

## 1.2. Objetivos

El objetivo principal de este trabajo es estudiar el funcionamiento, realizar experimentos y comparar diferentes herramientas de *testing* para *smart contracts*, concretamente Foundry y Echidna. Para cumplir con este objetivo se plantean los siguientes objetivos específicos:

- Estudiar el funcionamiento interno de las herramientas, y en particular la generación de secuencias con *mutation-based fuzzing* y los mecanismos auxiliares usados por las herramientas durante este proceso.
- Diseñar contratos inteligentes sencillos para poder realizar *tests* funcionales y de consumo de gas sobre ellos, y diseñar contratos inteligentes específicos que recreen la vulnerabilidad de reentrada. Los contratos sobre los que se realizarán los *tests* de consumo de gas deberán tener funciones de distinto coste computacional para poder estudiar distintos casos.
- Desarrollar *tests* para cada herramienta que comprueben que los contratos diseñados funcionan correctamente. Para evaluar la capacidad de las herramientas para detectar errores en los contratos se añadirá un error y se estudiará la información proporcionada por las herramientas.
- Desarrollar *tests* que permitan medir el gas consumido en funciones de distintas complejidades y estudiar mediante un *test* específico si las herramientas tienen en cuenta o no el coste de `calldata`.
- Desarrollar *tests* específicos para comprobar si las herramientas pueden o no detectar la vulnerabilidad de reentrada mediante la ejecución de dichos *tests*.
- Estudiar las ventajas y desventajas de las herramientas según los casos de uso probados y realizar una comparación entre ellas.

## 1.3. Plan de trabajo

Para poder alcanzar los objetivos descritos en el apartado anterior, el trabajo se organizó como sigue:

1. **Investigación de técnicas de *testing*:** Estudiar diferentes técnicas de *testing* de software y decidir cuál se usará en el trabajo.
2. **Investigación de herramientas de *testing*:** Investigar sobre herramientas que permitan hacer *testing* sobre contratos inteligentes y que implementen *mutation-based fuzzing* para poder realizar las pruebas.
3. **Instalación de las herramientas:** Configurar el entorno e instalar las herramientas seleccionadas y comprobar que las herramientas están bien instaladas.
4. **Análisis del funcionamiento interno de las herramientas:** Estudiar para cada herramienta el ciclo de *fuzzing*, cómo se generan las entradas y sus mecanismos auxiliares.

5. **Desarrollo de contratos inteligentes:** Desarrollar diferentes contratos inteligentes en cada herramienta para poder realizar las pruebas sobre estos. Para las pruebas funcionales y de consumo de gas los contratos serán sencillos. En cambio, para el estudio de detección de reentrada, el contrato deberá recrear esta vulnerabilidad.

6. **Desarrollo de *tests*:** Desarrollar *tests* que evalúen que las funciones de un contrato funcionan correctamente, midan cuánto gas consumen y que detecten o no la vulnerabilidad de reentrada.

7. **Evaluación de resultados:** Evaluar la capacidad de las herramientas de detectar fallos en un contrato, la capacidad de evaluar el consumo de gas de funciones y si la herramienta puede detectar reentrada. También se evalúa la información aportada por cada herramienta al realizar los *tests*.

8. **Redacción de la memoria:** Redactar la memoria de trabajo en la que se muestren las investigaciones hechas, los experimentos realizados y la comparación entre las herramientas.

## 1.4. Estructura de la Memoria

En este apartado se muestra un esquema de la estructura de los capítulos y secciones de la memoria:

En el Capítulo 1 se explica cuál es la motivación del proyecto, sus objetivos, qué plan de trabajo se ha seguido para realizar el proyecto y cómo está organizada la memoria. El Capítulo 2 describe brevemente qué es una plataforma *blockchain*, se presenta Ethereum y el *testing* de *software*. También explica conceptos más específicos del *testing* de *software*, en concreto las técnicas de *fuzzing*, *mutation testing* y *mutation-based fuzzing*. En el Capítulo 3 se analizan las herramientas Foundry y Echidna. Se explica cómo funcionan internamente y cómo generan nuevas secuencias con *mutation-based fuzzing*. En el Capítulo 4 se presentan los experimentos realizados con cada herramienta y se evalúan los resultados obtenidos. Se divide en tres partes:

- Evaluación funcional de los contratos: Se prueba la capacidad de las herramientas de detectar errores en un contrato.

- Evaluación del consumo de gas: Se evalúa la capacidad de cada herramienta para analizar el gas consumido por las funciones de un contrato. También se prueba si las herramientas tienen en cuenta `calldata` o no.

- Evaluación de reentrada: Se prueba si, mediante el desarrollo de un *test*, las herramientas son capaces de detectar la vulnerabilidad de reentrada.

En el Capítulo 5 se comparan las herramientas. Primero se describen individualmente cada una de las herramientas analizadas y luego se comparan entre sí. Por último, el Capítulo 6 resume las conclusiones del trabajo y se proponen líneas de trabajo futuro.

Capítulo 2

# Preliminares

Este capítulo presenta las nociones básicas necesarias para entender los detalles técnicos del proyecto realizado. En primer lugar se hace una introducción a la noción de plataforma *blockchain* (Sección 2.1) y se incide en el caso de Ethereum (Sección 2.2). Además se presentan los principales detalles técnicos y la novedad que introdujo esta plataforma de *blockchain* respecto a sus predecesores. Tras esto, se realiza una descripción de las principales técnicas de *testing* (Sección 2.3). Por último, se presentan algunas de las herramientas existentes para aplicar *testing* sobre contratos inteligentes (Sección 2.4).

## 2.1. Blockchain

Una plataforma *blockchain* es una base de datos distribuida que contiene el registro de un libro de contabilidad (*ledger*) en el que se registran las transacciones realizadas en una criptomoneda digital. Esta base de datos se almacena en un conjunto de ordenadores que forman una red. Estos ordenadores, llamados nodos, deben validar y aceptar todos los bloques de la cadena, lo que garantiza que todos los nodos mantienen la misma información. De esta forma, no se necesita ningún intermediario o nodo central para realizar las transacciones [38].

Una de las propiedades fundamentales de las plataformas *blockchain* es la inmutabilidad, que impide la manipulación del contenido de la base de datos. Los bloques de la *blockchain* están conectados entre sí, cada bloque tiene una referencia al bloque anterior formando una cadena [20]. Una vez se añaden los datos a la cadena, es muy costoso (computacional o económicamente) que se puedan modificar porque implicaría cambiar los bloques posteriores de la cadena y, además, se necesitaría el consenso de los demás nodos de la red [25].

La primera *blockchain* relevante fue Bitcoin, que implementa la criptomoneda bitcoin. A partir de estas ideas, han surgido nuevas plataformas *blockchain*, como Ethereum, que introdujo por primera vez la noción de *blockchain* programable con *smart contracts*, programas que se ejecutan de forma confiable dentro de la red de *blockchain* [38].

## 2.2. Ethereum

Ethereum es una plataforma *blockchain* descentralizada con una máquina virtual Turing-completa integrada llamada *Ethereum Virtual Machine* (EVM) que mantiene el estado de la red. Los nodos de la red guardan una copia del estado de la cadena. Si un usuario quiere que la EVM realice un cómputo o ejecute una transacción, el resto de nodos de la red deben verificarlo, validarlo y ejecutar el cálculo. Esto causa que el estado de la EVM cambie y se propague por todos los nodos de la red [20].

Los *smart contracts* son programas escritos en un lenguaje de alto nivel, que se ejecutan cuando son invocados desde una transacción [32]. El lenguaje de programación de contratos más utilizado en Ethereum es Solidity. Los contratos inteligentes son compilados a bytecode de la EVM, formado por un conjunto de instrucciones de máquina de bajo nivel [29].

El EVM bytecode es un lenguaje basado en pila, las instrucciones operan sobre una pila utilizada para almacenar valores de la ejecución [23]. Tiene un conjunto de instrucciones básicas que permiten realizar operaciones llamadas *opcodes*. Estos permiten realizar operaciones aritméticas (`ADD` o `MUL`), operaciones de manejo de bits (`AND` o `XOR`), operaciones para acceder a información del entorno de ejecución de Ethereum ( como por ejemplo la dirección del llamante con `CALLER` o la cantidad de gas restante con `GAS`), operaciones para calcular un código hash (`KECCAK256`) y operaciones para leer y escribir tanto en memoria como de `storage` (`MLOAD`, `MSTORE`, `SLOAD` o `SSTORE` respectivamente) [29].

En Ethereum se pueden distinguir 5 regiones de memoria: `stack` (pila), el componente principal donde se ejecutan la mayoría de cómputos; `memory`, una región de memoria volátil donde se almacenan temporalmente datos durante la ejecución de contratos; `storage`, una región de memoria persistente donde se almacena información que puede ser necesaria después de la ejecución de contratos, esta se encuentra replicada en todos los nodos de la red; `calldata`, una región solo de lectura, es donde se almacenan los parámetros de las transacciones; y `transient storage` una región de memoria volátil que persiste durante las llamadas que se puedan hacer al contrato en la misma transacción [27, 26].

Una transacción es una llamada a una función de un contrato desde una cuenta externa o EOA (*externally-owned account*) [23]. Cuando se quiere realizar una transacción es necesario pagar una tarifa llamada gas. Simplemente por realizar una transacción, ya se consumen 21000 unidades de gas, y si por ejemplo se quieren realizar operaciones como sumas (`ADD`), estas consumen 3 unidades de gas. Todas las instrucciones de la EVM consumen gas, por lo que si la transacción realizada es compleja, la tarifa de gas será mayor [20]. No solo las operaciones ejecutadas consumen gas: el uso de la memoria volátil (`memory`) tiene un coste que se incrementa cuadráticamente, y los parámetros con los que se realiza la transacción también. Estos son codificados en la región `calldata` y su coste depende del contenido de los datos, por cada byte igual a 0 se consumen 4 unidades de gas, mientras que para cada byte distinto de 0 se consumen 16 unidades [37].

## 2.3. Testing de software

El *testing* de *software* consiste en realizar pruebas sobre un programa con el objetivo de detectar errores, vulnerabilidades o comportamientos inesperados antes de que el *software* sea utilizado en un entorno real [14].

Las pruebas pueden clasificarse en dos tipos: manuales, los casos de prueba son diseñados a mano, y automáticas, los casos de prueba se diseñan automáticamente reduciendo posibles errores humanos en la creación de los casos [30].

En el caso de una plataforma *blockchain*, el *testing* es de vital importancia debido a la inmutabilidad de los contratos inteligentes. Una vez el contrato se despliega en la red no es posible modificarlo, por lo que cualquier error, vulnerabilidad o comportamiento inesperado del contrato puede ser explotado por atacantes [21].

Además, los contratos inteligentes gestionan transacciones económicas reales que pueden ser de gran valor. Por ello, si se despliega un contrato con vulnerabilidades, puede causar pérdidas significativas para los usuarios.

Uno de los casos más famosos ocurrió en 2016 con *The DAO*, un fondo de inversión construido en Ethereum. Un atacante explotó una vulnerabilidad de reentrada y consiguió sustraer aproximadamente 60 millones de dólares [12]. Fue un fallo tan grave que la SEC (Comisión de bolsa y valor de EE.UU.) tuvo que intervenir [33, pág. 9]. Finalmente la comunidad Ethereum se vio obligada a hacer un *Hard Fork* (una bifurcación de la red) para recuperar los fondos.

A continuación, se introducen los conceptos de *fuzzing* y *mutation testing*, ya que ambos sirven para entender el concepto de *mutation-based fuzzing* que es la técnica principal utilizada en este trabajo.

### 2.3.1. Fuzzing

El *fuzzing* es una de las técnicas de *testing* automático más utilizadas hoy en día. Consiste en ejecutar entradas sobre un programa con el objetivo de provocar fallos o vulnerabilidades. Estas entradas son generadas de forma automática y se utilizan como casos de prueba sobre el programa que se está probando [22].

Por ejemplo, si se realizara fuzzing sobre la función de la Figura 2.1, se llamaría una gran cantidad de veces a la función con entradas aleatorias, como por ejemplo (100, 8).

```
1 int example(int a, int b) {
2     if(a > b){
3         return a - b;
4     }
5     return a + b;
6 }
```

Figura 2.1: Función de ejemplo.

Para poder realizar esta técnica se utiliza un *fuzzer*, que es la herramienta que realiza *fuzzing* sobre un programa y registra los casos de prueba que produzcan

fallos. El proceso de *fuzzing* comienza con la generación de casos de prueba, estos pueden incluir entradas que el sistema no se espera. El *fuzzer* ejecuta estos casos sobre el programa y realiza un análisis del comportamiento y sobre los casos de prueba que han provocado errores.

### 2.3.2. Mutation Testing

Para poder evaluar la eficacia de un conjunto de pruebas se puede utilizar la técnica de *mutation testing*. Consiste en modificar el código del programa que se quiere probar y comprobar si los *tests* desarrollados detectan dichos fallos. Cada modificación realizada sobre el código recibe el nombre de mutante [28].

Las modificaciones que se suelen realizar son pequeñas y controladas. Tomando como ejemplo la función de la Figura 2.1, el *mutation testing* podría consistir en cambiar el operador "mayor que" por un "mayor o igual que". La idea del *mutation testing* es que un conjunto de pruebas robusto debería ser capaz de detectar los mutantes del código ya que modifican el comportamiento del programa analizado. Por lo que si las pruebas creadas para este contrato no detectan el mutante significa que pueden ser insuficientes o incompletas [28].

Sin embargo, en este trabajo no se aplican mutaciones sobre el contrato inteligente, sino sobre las entradas y secuencias de ejecución utilizadas en las pruebas.

### 2.3.3. Mutation-based Fuzzing

Aunque el *fuzzing* sea una técnica de *testing* efectiva, la generación de entradas es completamente aleatoria, lo que presenta limitaciones, ya que muchas entradas generadas pueden ser inválidas o pueden no aumentar la cobertura del programa.

Para mejorar la técnica de *fuzzing* tradicional surge el *mutation-based fuzzing*, una técnica que genera nuevos casos de pruebas a partir de mutaciones de entradas válidas. Las mutaciones que se realizan pueden aplicarse sobre diferentes elementos, como los parámetros de entrada de las funciones, secuencias de llamadas o transacciones ejecutadas. A partir de estas mutaciones, el sistema busca ampliar la cobertura de código y permitir la exploración de gran parte del flujo del programa [17].

El proceso comienza con la ejecución de un conjunto de entradas válidas para el programa. A partir de estas primeras entradas se aplican mutaciones, como reordenación de secuencias o modificaciones en los parámetros de las funciones para generar nuevos casos de prueba e intentar ampliar la cobertura de código. Cuando una secuencia es considerada interesante, es decir, ha ampliado la cobertura alcanzada, esta se almacena en el *corpus*, que es el conjunto de entradas que el *fuzzer* reutiliza para generar futuras iteraciones. De esta forma el conjunto de entradas evoluciona haciendo que las siguientes mutaciones sobre los casos de prueba sean más efectivas. Este proceso se repite de manera continua y automática, explorando distintas rutas de ejecución del programa.

Si se realizara *mutation-based fuzzing* sobre la función de la Figura 2.1, se generaría una primera entrada aleatoria sobre la función y, como se ha aumentado la cobertura de código, esta entrada será almacenada por el *fuzzer*. Para la siguiente

iteración de la campaña de *mutation-based fuzzing*, el *fuzzer* mutará esta entrada y volverá a ejecutar la función. Cuando se cumpla la condición del `if` se habrá aumentado la cobertura de código, por lo que la entrada que ha satisfecho la condición será guardada y será usada para las siguientes mutaciones.

## 2.4. Testing para contratos inteligentes

Las características de inmutabilidad y ejecución limitada por el gas de los contratos inteligentes, hacen que las herramientas de *testing* convencionales puedan no ser suficientes para realizar pruebas sobre ellos. Para solucionar este problema, se han desarrollado diferentes herramientas que permiten realizar pruebas sobre los contratos. Muchas de estas se basan en el *property-based testing*, que consiste en definir propiedades de un programa. Herramientas como Echidna [35] o Foundry [1] permiten ejecutar este tipo de *tests* mediante la técnica de *mutation-based fuzzing*. Aunque no es el único tipo de *testing* que ofrecen estas herramientas, ya que también se pueden realizar *tests* específicos para el funcionamiento de funciones concretas de un contrato, o realizar reportes de gas, en el caso de Foundry [9]. A diferencia de estas herramientas, Medusa [2] no presenta un enfoque centrado en propiedades, se usa más como *fuzzer* para realizar pruebas sobre contratos con la técnica de *mutation-based fuzzing*. Sin embargo, esta herramienta también resulta de utilidad, al igual que Foundry y Echidna, para pruebas de funcionalidad o de consumo de gas. Otra herramienta de utilidad es SolAR [18] que permite generar *tests* para contratos inteligentes enfocados en la cobertura de código.

Aparte de realizar *testing* sobre el contrato, también se puede comprobar si los *tests* creados son suficientes para probar el contrato. Herramientas como SuMo [4, 11] permiten realizar *mutation testing* en Solidity. SuMo implementa 44 operadores de mutación y podría modificar el código fuente de la Figura 2.1 como se ha mencionado anteriormente en el apartado de *mutation testing*.

Capítulo 3

# Herramientas y Análisis de Arquitectura Interna

Este capítulo presenta las herramientas estudiadas y su proceso de generación de entradas. En primer lugar, se describe las herramientas por las que está formado Foundry (Sección 3.1) y cuál es su funcionamiento. Se analiza el bucle de *fuzzing* de la herramienta y el proceso de generación de secuencias. Por último, se introduce la herramienta Echidna (Sección 3.2) explicando su funcionamiento, los distintos modos de *testing* que ofrece y su bucle de *fuzzing*. También se analiza el proceso de generación de secuencias de la herramienta y el proceso de *shrinking*.

## 3.1. Foundry

Foundry [1] es un entorno de desarrollo y pruebas para contratos inteligentes en Solidity. Está escrito íntegramente en Rust y emplea revm [3], una implementación de la EVM en dicho lenguaje, como motor de ejecución. Esta decisión de diseño tiene un impacto directo en el rendimiento del *fuzzer*, ya que Foundry no depende de una instancia externa, la ejecución de cada transacción se realiza dentro del mismo proceso, lo que elimina la sobrecarga asociada a la comunicación entre procesos y a la serialización y deserialización de datos.

Foundry incorpora un enfoque *mutation-based fuzzing* ya que almacena casos de prueba que han aumentado la cobertura y los muta para hacer el proceso de *fuzzing* más efectivo.

### 3.1.1. Ejecución de tests

Foundry se compone principalmente de cuatro herramientas [1]:

- **Forge:** Es la herramienta principal de Foundry. Permite compilar, probar, realizar *fuzzing* y desplegar contratos inteligentes escritos en Solidity [8].
- **Cast:** Puede realizar llamadas RPC a Ethereum (llamadas a procedimiento remoto), envía transacciones y decodifica datos desde la línea de comandos [6].

- **Anvil:** Nodo local de Ethereum rápido para desarrollo y pruebas. Se ejecuta en memoria y permite realizar *forking* desde cualquier cadena compatible con la EVM [5].

- **Chisel:** Es un bucle de lectura, evaluación e impresión interactivo para Solidity. Está diseñado para la creación rápida de prototipos y la depuración. Permite escribir y ejecutar código Solidity al instante sin necesidad de compilar contratos complejos [7].

El funcionamiento de Foundry se basa en generar y ejecutar casos de prueba en los *tests* creados y, cuando se terminen de ejecutar los *tests*, muestra por terminal los resultados obtenidos.

Foundry ejecuta cada *test* hasta que este falla o se alcanza el límite de *fuzzing* (256 casos por defecto). En algunos casos, aunque es menos común que ocurra, se dejarán de ejecutar los *tests* si se excede el gas límite de ejecución o se alcanza el límite de tiempo.

#### 3.1.1.1. Inicialización del fuzzing

Cuando el usuario ejecuta `forge test`, se organiza la ejecución de los *tests* de la siguiente forma:

1. Se localizan todos los contratos con *tests* definidos y se ejecutan en paralelo.

2. Para cada contrato se realiza una fase de preparación en la que se despliega su estado inicial en la EVM y se ejecuta, si existe, la función de inicialización de cada contrato. Se identifican las funciones que corresponden a *tests*.

3. Los *test* se ejecutan de forma individual y, durante su ejecución, se determina el tipo de *test* que representa. El *test* puede ser de los siguientes tipos: `UnitTest`, pruebas unitarias con entradas fijas, `FuzzTest`, *test* en el que Foundry genera automáticamente entradas aleatorias, o `InvariantTest`, *test* que verifica que el contrato cumple una propiedad a lo largo de toda la ejecución del programa.

4. Para los *tests* de *fuzzing* se instancia un `FuzzedExecutor` que contiene el bucle principal de *fuzzing* y el `CorpusManager`.

Además, se crea un vector llamado `history_map` para registrar las partes del código ejecutadas durante el *fuzzing*. Cada posición de este vector registra el *hitcount* binario de una arista del grafo de flujo de control del contrato (CFG), es decir, una representación simplificada del número de veces que dicha arista ha sido ejecutada.

A continuación, se inicializa el *corpus* donde se almacenarán secuencias de transacciones que hayan aumentado la cobertura de código. Se comprueba la dirección del *corpus* para ver si este existe o no. Si existe, este se carga y se reproduce en la EVM para modificar el `history_map` registrando las aristas ya visitadas. En caso contrario, el sistema recurrirá a la generación aleatoria de secuencias para la construcción progresiva del *corpus* durante la ejecución. A medida que se vayan añadiendo secuencias al corpus se harán mutaciones sobre ellas.

#### 3.1.1.2. Fuzzing

Una vez inicializado el *corpus*, se comienza el bucle principal de *fuzzing* en el que se toma la decisión central de explorar o explotar.

- Si el *corpus* en memoria está vacío, se genera una nueva transacción aleatoria (exploración).
- En caso contrario, se selecciona una entrada del *corpus* como principal y otra como secundaria para las mutaciones que las requieran (explotación). A partir de estas entradas se generan nuevas secuencias mediante operadores de mutación.

En ambos casos, la secuencia generada se ejecuta. En función de los resultados, el *corpus* puede actualizarse incorporando secuencias que hayan ampliado la cobertura del código.

Tras cada ejecución en la EVM, el sistema evalúa si la secuencia generada ha alcanzado partes del código del contrato que nunca antes habían sido ejecutadas. Esta evaluación determina si la entrada merece ser conservada en el *corpus* para futuras mutaciones. La función `merge_edge_coverage()` compara la cobertura obtenida durante la ejecución con el `history_map`. Durante esta comprobación puede haber dos tipos de descubrimientos:

- **Nueva arista:** La ejecución transitó por una arista del CFG que nunca había sido visitada.
- **Nueva *feature*:** Una arista ya conocida fue visitada un número de veces de tal manera que pertenece a un nuevo rango de *hitcount*.

Esta información permite al *fuzzer* aprender no solo si un camino es alcanzable, sino también con qué frecuencia. El vector `history_map` acumula el historial de todo el ciclo de vida de la campaña. No se resetea entre iteraciones, garantizando que el *corpus* no retroalimente entradas que describen cobertura ya conocida. Una vez analizada la cobertura, se decide cómo gestionar la secuencia ejecutada: Si la secuencia produce nueva cobertura se llevan a cabo tres acciones:

1. Se actualizan las estadísticas de la secuencia principal que generó la mutación (`total_mutations` y `new_finds_produced`).
2. La nueva secuencia se guarda en disco dentro del directorio de *corpus*.
3. La secuencia se añade al *corpus* en memoria para que pueda reutilizarse y mutarse en futuras iteraciones.

Por el contrario, si la secuencia no produce nueva cobertura esta no se añade al *corpus*. Sin embargo, el sistema sí actualiza las estadísticas de la secuencia principal incrementando su contador `total_mutations`.

Foundry también implementa un mecanismo de gestión del tamaño del *corpus*. Para que se elimine una secuencia del *corpus* debe cumplirse que el tamaño del *corpus* en memoria supere `corpus_min_size` y que la entrada haya sido

mutada más de `corpus_min_mutations` veces y no tiene el estado `is_favored`. La variable `is_favored` de una secuencia del *corpus*, se activa cuando el cociente `new_finds_produced` / `total_mutations` supera `FAVORABILITY_THRESHOLD` (0,3, l. 28). Las entradas con `is_favored = true` son inmunes a ser eliminadas independientemente de cuántas veces hayan sido mutadas.

#### 3.1.1.3. Generación de secuencias

Para la generación de secuencias se combinan tres niveles de mutación: a nivel de secuencia, a nivel de parámetros y operadores matemáticos de bajo nivel.

Las mutaciones a nivel de secuencia actúan sobre la secuencia completa de transacciones sin modificar los argumentos individuales de cada llamada. Se pueden realizar las siguientes mutaciones:

- **Splice:** Toma un fragmento aleatorio de la secuencia primaria y lo concatena con un fragmento aleatorio de la secuencia secundaria. Genera secuencias que combinan rutas de ejecución que, por separado, ya han demostrado ser interesantes.
- **Repeat:** Selecciona una única transacción de la secuencia y la repite varias veces consecutivas, sustituyendo un segmento aleatorio de la secuencia original.
- **Interleave:** Combina en paralelo las dos secuencias usando `zip`: por cada par de transacciones en la misma posición, elige aleatoriamente una de las dos. Genera secuencias que intercalan decisiones de dos caminos de ejecución diferentes.
- **Prefix y Sufix:** Ambos operadores parten de una copia de una secuencia existente y regeneran aleatoriamente una porción de ella. `Prefix` sustituye los primeros n elementos y `Suffix` sustituye los últimos n elementos. Esto preserva parte del historial que ya ha demostrado ser interesante, al mismo tiempo que inyecta una variación al principio o final de la secuencia.
- **Abi:** Selecciona aleatoriamente una transacción dentro de la secuencia y muta sus argumentos ABI, sin alterar el orden ni la composición de la secuencia. El ABI (*Aplication Binary Interface*) es la interfaz que define cómo se codifican y decodifican las secuencias de un contrato, incluyendo sus parámetros y tipos de datos.

El funcionamiento de la mutación a nivel de parámetros, es el siguiente:

1. Se decodifica el `calldata` actual, excluyendo el selector de la función (4 bytes iniciales), obteniendo los parámetros originales mediante la función `abi_decode_input()`.
2. Se determina aleatoriamente cuántos argumentos mutar en esta iteración: entre 1 y el número total de parámetros de la función.
3. Para cada argumento seleccionado se invoca `mutate_param_value()` que mutará el valor del argumento.

4. Se recodifican los argumentos mutados y se sobreescriben los parámetros originales con los nuevos valores codificados.

A diferencia de la mutación a nivel de parámetros descrita anteriormente, que decide qué parámetros mutar dentro de una llamada, la mutación a nivel de operadores matemáticos de bajo nivel determina cómo se modifica cada valor dependiendo del tipo de dato. La función `mutate_param_value()` actúa como enrutador: inspecciona el tipo del argumento y lo delega en el operador matemático correspondiente. Si el operador seleccionado devuelve `None` (por ejemplo, por no encontrar un valor diferente al original), recurre a generar un valor a partir el diccionario de estado, que es donde se almacenan valores e *inputs* previamente observados y utilizados durante la ejecución del *fuzzing*.

Dentro de este nivel se utilizan los siguientes operadores de mutación:

- **Valores de interés:** Sustituye el valor seleccionado por valores que históricamente provocan fallos en implementaciones de *software*: límites de tipos, potencias de dos y cero.
- **Bit Flip:** Invierte un bit aleatorio.
- **Ruido Gaussiano:** En lugar de cambiar drásticamente el valor, este mutador aplica una perturbación proporcional centrada en el valor original. El factor de escala se genera mediante el método de Irwin-Hall [34]: suma de 8 muestras uniformes en $[0, 1)$, que aproxima una distribución normal, centrada en 1,0 y escalada con uno de los multiplicadores `THREE_SIGMA_MULTIPLIERS` = {0,1; 0,25; 0,5; 1,0; 2,0; 5,0; 10,0}.
- **Valores límite:** Genera un valor aleatorio dentro de un rango $[\texttt{min}, \texttt{max}]$ mediante el operador módulo, garantizando que el resultado sea diferente que el valor original.
- **Increment/Decrement:** Suma o resta 1 al valor y valida que el resultado sea diferente y pueda ser representado usando el tipo del valor original. Es el operador ideal para detectar errores *off-by-one* y condiciones de frontera en bucles y comparaciones.

## 3.2. Echidna

Echidna es una herramienta desarrollada en Haskell orientada al *fuzzing* de contratos inteligentes con un enfoque basado en propiedades. Su objetivo es violar las propiedades o invariantes definidas por el usuario sobre el contrato mediante la generación de distintas secuencias de llamadas a las funciones públicas del contrato con entradas aleatorias [24].

Para ello, Echidna usa información derivada del ABI (*Application Binary Interface*) del contrato, lo que permite conocer su interfaz pública y construir campañas de *fuzzing* estructuradas en lugar de simples entradas aleatorias [35].

Además, Echidna incorpora un enfoque *mutation-based fuzzing*. Al ejecutar las pruebas, se guardan en el *corpus* los conjuntos de secuencias que han permitido

explorar nuevos estados del contrato o han aumentado la cobertura. Estas secuencias se reutilizan mutándose para generar nuevas variaciones con el objetivo de tener una mejor cobertura del código y descubrir posibles violaciones.

### 3.2.1. Ejecución de tests

Como se ha mencionado anteriormente, la ejecución de *tests* en Echidna se basa en generar de forma continua secuencias de llamadas con distintos parámetros y, tras cada secuencia, analizar que los *tests* se cumplen. Este proceso se mantiene hasta que todas las propiedades o *tests* fallan (depende del modo de *testing*), se llega al límite de llamadas (parámetro `testLimit`, por defecto 50000), o agotamiento de tiempo.

Echidna ofrece distintos modos de *testing* que permiten adaptar el proceso de *fuzzing* a los objetivos del usuario. El modo de *testing* se puede indicar en el archivo de configuración o con la opción `-test-mode` en la línea de comandos. Se pueden diferenciar los siguientes modos [35]:

- **Property**: Es el modo por defecto, se prueban las propiedades definidas en los *tests* por el usuario. Estas son funciones sin parámetros prefijadas con `echidna_` que devuelven **bool**. Este es un modo útil para comprobar invariantes.
- **Assertion**: La herramienta busca detectar fallos de aserción (`assert()`). Si la condición del `assert` falla, Echidna registra la secuencia como un fallo.
- **Foundry**: Permite ejecutar *tests* de estilo Foundry, funciones prefijadas con `test` o `invariant_` definidas.
- **Overflow**: Busca desbordamientos de `integer`. Intenta forzar combinaciones de entradas y estados que hagan que una operación aritmética rebase los límites de un tipo (para versiones de Solidity a partir de la 0.8.0).
- **Optimization**: busca secuencias de transacciones que maximizan el valor de retorno de las funciones con prefijo `echidna_` que devuelven `int256`, utilizando dicho valor como criterio de evaluación para guiar la búsqueda de estados con mayor valor.
- **Exploration**: Se ejecuta sin comprobar las propiedades, se centra en recoger información sobre cobertura.

Echidna también implementa mecanismos adicionales que mejoran la eficacia del proceso de *testing*. Entre ellos se encuentran la implementación de workers, que paraleliza la ejecución de los *tests*, la gestión de un *corpus*, que reutiliza secuencias que han resultado interesantes, y el proceso de *shrinking*, que simplifica los casos de prueba que han dado error.

El funcionamiento de Echidna se basa en un ciclo continuo de generación, ejecución y evaluación de casos de prueba. La ejecución se detiene cuando todos los *tests* o propiedades han sido falsificados, o cuando se alcanza alguno de los límites configurados, como el número máximo de ejecuciones (`testLimit`) o el tiempo máximo de ejecución establecido.

#### 3.2.1.1. Inicialización del fuzzing

El proceso de *fuzzing* comienza con la selección de una semilla que sirve como punto de partida para la generación de secuencias aleatorias.

A continuación, se llama a una función que prepara el entorno de Echidna antes de realizar *fuzzing* sobre un contrato. Esta función devuelve la máquina virtual EVM, el estado global de la campaña de *fuzzing*, y un diccionario de funciones que permite que la herramienta conozca los nombres de las funciones, los tipos de parámetros que tienen y pueda generar llamadas válidas. Durante esta fase también se carga el *corpus* inicial utilizado por el *fuzzer*. El usuario puede proporcionar un directorio de *corpus* para que la herramienta reutilice dichas secuencias para la exploración del código.

La carga del *corpus* se realiza mediante la función `loadInitialCorpus`, utilizada durante la construcción de la campaña de *fuzzing*. Si no se indica un directorio el *corpus* será vacío, si se indica se cargará el *corpus* predefinido por los ficheros presentes en ese escritorio.

#### 3.2.1.2. Fuzzing

Para realizar el proceso de *fuzzing*, Echidna usa *workers*, procesos paralelos que ejecutan el bucle principal de *fuzzing* de forma independiente y comparten resultados mediante el *corpus*.

Antes de comenzar el *fuzzing*, el *corpus* inicial se divide entre los *workers* para favorecer que cada proceso explore distintas rutas de ejecución del contrato. Esta división se realiza únicamente en la primera fase de la ejecución permitiendo que se mejore la eficiencia del proceso de *fuzzing*. Una vez inicializados, cada *worker* realiza el bucle principal de *fuzzing*, que consiste en decidir continuamente que acción realizar sobre los casos de prueba disponibles:

- Si hay un *test* que haya fallado y puede reducirse, se realiza *shrinking* sobre él. Este proceso consiste en simplificar iterativamente la secuencia de transacciones y comprobar tras cada simplificación que la secuencia sigue provocando fallo. Se hablará en más detalle de este proceso en la sección 3.2.1.4.
- Si no hay *tests* sobre los que se pueda realizar *shrinking*, se continúa con el proceso de *fuzzing*.
- Si no queda trabajo pendiente, el *worker* finaliza su ejecución.

Durante el proceso de *fuzzing*, Echidna selecciona una secuencia de transacciones del *corpus*, la muta y ejecuta dicha secuencia. Después de la ejecución, Echidna analiza los resultados y actualiza el *corpus* en caso de ser necesario.

Cuando una secuencia produce nueva cobertura, esta se considera interesante. La variable `newCoverage` pasa a valer `true` y se actualiza el *corpus* almacenando esta nueva secuencia. Al añadir una nueva secuencia al *corpus* se le asigna un peso, que corresponde al número de secuencia (`ncallseqs + 1`), de modo que será seleccionada con mayor probabilidad para futuras generaciones de nuevas secuencias. La variable `ncallseqs` es un contador interno de Echidna que cuenta cuántas secuencias de transacciones se han ejecutado durante la campaña de *fuzzing*.

#### 3.2.1.3. Generación de secuencias

Para la generación de secuencias Echidna combina dos niveles de mutación: a nivel de transacciones y a nivel de secuencias completas. Las mutaciones a nivel de transacciones se realizan sobre un trozo de la secuencia y pueden ser las siguientes:

- **Identity:** No modifica nada, permite reaprovechar la secuencia tal cual.
- **Mutation:** Cambia aleatoriamente los argumentos de las llamadas.
- **Shrinking:** Simplifica los parámetros de las transacciones.
- **Expansion:** Se añaden nuevas transacciones al final de la secuencia, seleccionadas a partir del conjunto de funciones del contrato.
- **Swapping:** Se reordenan las transacciones existentes en la secuencia.
- **Deletion:** Se eliminan algunas transacciones.

Las mutaciones a nivel de secuencias completas pueden ser las siguientes:

- **RandomAppend:** El trozo de la secuencia mutada se inserta al inicio de dicha secuencia.
- **RandomPrepend:** El trozo de la secuencia mutada se inserta en una parte intermedia de la secuencia.
- **RandomSplice:** Combina dos secuencias del *corpus* copiando y pegando partes de ambas.
- **RandomInterleave:** Entrelaza las transacciones de dos secuencias del *corpus*.

Si el *corpus* está vacío se generará una secuencia aleatoria. Por el contrario, si no está vacío se mutará una secuencia del *corpus* siendo `seqLen` la longitud objetivo de la nueva secuencia. La generación de secuencias comienza con la elección de la estrategia de mutación. Si la longitud de secuencia es mayor que uno se usará `seqMutatorStateful` que contempla todas las combinaciones posibles (`RandomAppend` y `RandomPrepend` con `Identity`, `Shrinking`, `Mutation`, `Expansion`, `Swapping` y `Deletion`, además de `RandomSplice` y `RandomInterleave`). Si la longitud de secuencia es 1 se usará `seqMutatorStateless`, una versión más simple, solo considera `RandomAppend` y `RandomPrepend` con `Identity`, `Shrinking` y `Mutation`. Ambas funciones devuelven un `CorpusMutation` eligiendo entre los distintos tipos de mutación. Cuando `CorpusMutation` es de la forma `RandomAppend` o `RandomPrepend` se realiza una mutación de una sola secuencia:

1. **Selección de la secuencia del corpus:** Se elige una secuencia del *corpus* para mutar, de forma ponderada. De esta forma las secuencias añadidas más tarde, y normalmente más productivas, tienen más probabilidades de ser seleccionadas, como se ha explicado anteriormente con la variable `ncallseq`.
2. **Corte parcial de la secuencia:** Se elige un punto de corte `k` al azar en la secuencia para que este sea mutado.

3. **Mutación de la porción de la secuencia:** Se aplica la mutación sobre el trozo de secuencia, esta mutación puede ser de cualquiera de las del nivel de transacción.

4. **Inserción en la secuencia base:** En `RandomAppend` el trozo se concatena al principio de la secuencia, y con `randomPrepend` se concatena en una posición `k` aleatoria dentro de la secuencia. En ambos casos, el resultado se recorta a `seqLen` para mantener la longitud de secuencia deseada.

En cambio, cuando `CorpusMutation` es de la forma `RandomSplice` o `Random Interleave` se cambia el enfoque a la combinación de dos secuencias:

1. **Selección de dos secuencias del corpus:** Se seleccionan dos secuencias del *corpus* para proceder a la combinación de estas.

2. **Combinación:** Se combinan las secuencias, si la opción es `RandomSplice` se mezclan fragmentos de ambas secuencias para crear una nueva, y si la opción es `RandomInterleave` se intercalan los elementos de ambas secuencias.

3. **Inserción en la secuencia base:** La secuencia generada se integra con la secuencia base y se ajusta a `seqLen` para mantener la longitud de secuencia deseada.

Después de haber generado la secuencia esta se ejecuta y se analizan los resultados, comprobando si introduce nueva cobertura o falsifica algún *test*.

#### 3.2.1.4. Proceso de shrinking

Cuando un *test* falla, Echidna inicia un proceso de *shrinking* que consiste en reducir las secuencias que hacen que una propiedad o *test* se falsifique, transformándolas en casos de uso mínimos reproducibles. El *shrinking* se ejecuta hasta alcanzar el *shrinkLimit* (parámetro configurable cuyo valor por defecto es 5000 iteraciones) o hasta obtener el *test case* mínimo reproducible que aún falsifique la propiedad o el *test*. El ciclo es el siguiente:

1. `removeReverts`**:** Reemplaza las transacciones revertidas por `NoCall`, que actúa como una llamada vacía que no ejecuta ninguna función del contrato ni modifica el estado.

2. `removeUselessNoCalls`**:** Elimina los `NoCall` sin cambios de estado.

3. `shrinkSeq`**:** Elige aleatoriamente (50 %) entre:

   - Shorten: Elimina una transacción aleatoria de la secuencia.
   - Shrunk: Simplifica los argumentos de las llamadas.

4. `checkETest` **(verificación):** Después de reducir la secuencia esta se re-ejecuta para comprobar que la propiedad o el *test* siga fallando, en este caso la nueva secuencia se acepta, en caso contrario se descarta.

El proceso se realiza de forma iterativa, evaluando en cada paso si la secuencia reducida sigue reproduciendo el fallo.

Capítulo 4

# Casos de Uso e Implementación

En este capítulo se presentan los experimentos realizados con cada una de las herramientas. Primero se realiza una evaluación funcional de un contrato con las herramientas (Sección 4.1), en esta sección se introduce un error en el contrato para comprobar la capacidad de las herramientas de detectarlo. En segundo lugar se realiza una evaluación de consumo de gas de las funciones de un contrato (Sección 4.2). Se analiza el resultado de la evaluación obtenido con cada herramienta y se hacen pruebas para comprobar si la herramienta tiene en cuenta `calldata`. Por último se comprueba la capacidad de la herramienta de detectar la vulnerabilidad de reentrada (Sección 4.3). Se recrea la vulnerabilidad de reentrada y, mediante unos *tests*, se comprueba si la herramienta es capaz de detectarla.

## 4.1. Evaluación funcional de los contratos

El objetivo de la evaluación funcional de los contratos es verificar que las operaciones implementadas en el contrato se comportan según su lógica esperada. Para ello, se analiza el comportamiento del contrato mediante *tests* que aplican la técnica de *mutation-based fuzzing*, comprobando que el estado del contrato evoluciona correctamente cuando las operaciones son válidas.

### 4.1.1. Diseño del contrato de prueba

Para realizar la evaluación funcional se ha diseñado un contrato sencillo denominado `Counter`. Este contrato mantiene una variable de estado de tipo `uint16` llamada `number` que representa un contador cuyo valor puede modificarse mediante diferentes funciones. Esta variable es pública para facilitar la verificación de los *tests*.

Aunque el contrato presenta una complejidad reducida, incorpora elementos adecuados para evaluar técnicas de *mutation-based fuzzing*:

- Operaciones aritméticas con riesgo potencial de desbordamiento.
- Bucles simples y anidados dependientes de parámetros externos.
- Funciones que modifican estado.

El uso de los bucles simples y anidados permite generar múltiples rutas de ejecución cuya exploración depende de los valores de entrada. El contrato se muestra en la Figura 4.1.

```
1  // File: src/Counter.sol
2  // SPDX-License-Identifier: UNLICENSED
3  pragma solidity 0.8.13;
4
5  contract Counter {
6      uint16 public number;
7
8      function setNumber(uint16 newNumber) public {
9          number = newNumber;
10     }
11
12     function increment(uint16 times) public {
13         require(uint256(number) + uint256(times) <= type(uint16).
               max, "Overflow");
14
15         for (uint16 i = 0; i < times; i++) {
16             number++;
17         }
18     }
19
20     function double(uint16 a, uint16 b) public {
21         uint256 total = uint256(a) * uint256(b);
22         require(total <= type(uint16).max, "Overflow");
23         require(uint256(number) + total <= type(uint16).max, "
               Overflow");
24
25         for (uint16 i = 0; i < a; i++) {
26             for (uint16 j = 0; j < b; j++) {
27                 number++;
28             }
29         }
30     }
31 }
```

Figura 4.1: Código fuente del contrato utilizado para las pruebas funcionales.

El contrato incluye varias operaciones que permiten modificar el valor del contador mediante distintas funciones:

- La función `setNumber` permite asignar directamente un nuevo valor a la variable `number`.
- La función `increment` incrementa el valor del contador un número determinado de veces, indicado por parámetro, mediante un bucle.
- La función `double` realiza incrementos haciendo uso de dos bucles anidados.

Las funciones `increment` y `double` incluyen comprobaciones mediante la instrucción `require` para evitar desbordamiento de la variable de tipo `uint16`. Estas comprobaciones comprueban que el resultado de las operaciones no supere el valor máximo de `uint16`, en este caso la función se revierte.

### 4.1.2. Diseño del conjunto de pruebas

El conjunto de pruebas se ha diseñado con el objetivo de verificar el correcto comportamiento de las funciones del contrato. Debido a las particularidades que presenta cada herramienta, se han ajustado los conjuntos de pruebas a cada una de ellas.

Los *tests* desarrollados para la herramienta Foundry, mostrados en la Figura 4.2, se basan en ejecuciones en las que el *fuzzer* genera valores aleatorios para los valores de entrada de cada *test*. Esto permite verificar los resultados de funciones concretas.

Los *tests* desarrollados para Echidna se encuentran en la Figura 4.3. Esta herramienta tiene un enfoque basado en propiedades donde, por defecto, los *tests* son invariantes del contrato que deben cumplirse para cualquier secuencia de llamadas. Ajustando el modo de *testing* a `assertion`, se puede verificar individualmente el comportamiento de cada función mediante aserciones (`assert()`).

Además de realizar *mutation-based fuzzing* los parámetros de las funciones, Echidna construye secuencias de llamadas en distinto orden. En este caso, como lo que se quiere comprobar es el correcto funcionamiento de unas funciones en concreto y no comprobar que el contrato en general cumpla ciertas propiedades, los *tests* se han desarrollado de manera que Echidna realiza *fuzzing* sobre los parámetros del *test*, se llama a la función que se quiere probar y se comprueba que la salida de la función es correcta.

En lugar de comprobar resultados concretos, los *tests* validan relaciones matemáticas que deben cumplirse para cualquier combinación válida de parámetros, por ejemplo, en la función `increment` el *test* verifica que el valor final de la variable `number` es igual al valor inicial de dicha variable más el número de iteraciones.

### 4.1.3. Evaluación por herramienta

Una vez definido el contrato de prueba y el conjunto de pruebas, se procede a ejecutar dichas pruebas utilizando Foundry y Echidna.

Cada una de estas herramientas ofrece distinta información de salida de los *tests*.

También se introducirá una modificación intencionada para evaluar la capacidad de las herramientas para detectar errores en el contrato. La modificación se realizará en la función `increment` añadiendo la siguiente línea de código dentro del bucle `for`:

```
if (number % 7 == 0 && number != 0) number++;
```

En las siguientes subsecciones se analizan los resultados obtenidos con cada una de las herramientas.

```
1  // SPDX-License-Identifier: UNLICENSED
2  pragma solidity 0.8.13;
3
4  import "forge-std/Test.sol";
5  import "../src/Counter.sol";
6
7  contract CounterTest is Test {
8      Counter public counter;
9
10     function setUp() public {
11         counter = new Counter();
12     }
13
14     function testSetNumber(uint16 num) public {
15         counter.setNumber(num);
16         assertEq(counter.number(), num);
17     }
18
19     function testIncrement(uint16 num, uint16 times) public {
20         counter.setNumber(num);
21
22         if(uint256(num) + uint256(times) > type(uint16).max){
23             vm.expectRevert(bytes("Overflow"));
24             counter.increment(times);
25         }else{
26             counter.increment(times);
27             assertEq(counter.number(), num + times);
28         }
29     }
30
31     function testDouble(uint16 num, uint16 a, uint16 b) public {
32         counter.setNumber(num);
33
34         uint256 mult = uint256(a) * uint256(b);
35
36         if(mult > type(uint16).max){
37             vm.expectRevert(bytes("Overflow"));
38             counter.double(a, b);
39         }else if(uint256(num) + mult > type(uint16).max){
40             vm.expectRevert(bytes("Overflow"));
41             counter.double(a, b);
42         }else{
43             counter.double(a, b);
44             assertEq(counter.number(), uint16(num + mult));
45         }
46     }
47 }
```

Figura 4.2: Código fuente de las pruebas funcionales para Foundry.

#### 4.1.3.1. Foundry

Para ejecutar las tres pruebas que verifican el comportamiento del contrato se utiliza el comando `forge test`.

```
1  pragma solidity 0.8.34;
2
3  import "../contracts/Counter.sol";
4
5  contract CounterEchidnaTest is Counter{
6
7      function testSetNumber(uint16 num) public{
8          setNumber(num);
9          assert(number == num);
10     }
11
12     function testIncrement(uint16 num, uint16 times) public{
13         setNumber(num);
14         increment(times);
15
16         assert(number == num + times);
17     }
18
19     function testDouble(uint16 num, uint16 a, uint16 b) public{
20         uint256 total = uint256(a) * uint256(b);
21
22
23         setNumber(num);
24         double(a, b);
25
26         assert(number == num + uint16(total));
27     }
28 }
```

Figura 4.3: Código fuente de las pruebas funcionales para Echidna.

La salida proporcionada por Foundry, mostrada en la Figura 4.4, nos permite saber si el *test* ha pasado o no y, además, incluye información adicional, como el número de ejecuciones realizadas por el *fuzzer* representado por el valor "runs", el consumo medio de gas durante las ejecuciones representado por el valor $\mu$, y la mediana de gas consumido durante la ejecución representado por el valor ~.

Como puede observarse, todas las pruebas se han ejecutado correctamente sin fallos, lo que significa que las funciones del contrato se comportan de acuerdo a lo esperado. Cada *test* se ejecuta múltiples veces con diferentes valores de entradas generados automáticamente por el *fuzzer*.

Tras modificar la función `increment` y volver a ejecutar las pruebas, se muestra en la Figura 4.5 el análisis realizado por Foundry.

En la salida se observa como el *test* asociado a la función `increment` ha fallado ya que la función ha devuelto 10750 y el valor esperado era 9215. Foundry también indica con qué entrada ocurre este error, en este caso cuando los valores son `num = 1` y `times = 9214`.

Los resultados obtenidos muestran que Foundry es capaz de detectar el error introducido en el contrato, y que la información proporcionada es suficiente para reproducir el error.

```
[⠊] Compiling...
No files changed, compilation skipped

Ran 3 tests for test/Counter.sol:CounterTest
[PASS] testDouble(uint16,uint16,uint16) (runs: 256, μ: 1830282, ~: 34165)
[PASS] testIncrement(uint16,uint16) (runs: 257, μ: 2650484, ~: 72082)
[PASS] testSetNumber(uint16) (runs: 256, μ: 23399, ~: 29463)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.32s (2.32s CPU time)

Ran 1 test suite in 1.33s (1.32s CPU time): 3 tests passed, 0 failed, 0 skipped (3 total tests)
```

Figura 4.4: Resultado de los *tests* de funcionalidad de Foundry.

```
[⠊] Compiling...
No files changed, compilation skipped

Ran 3 tests for test/Counter.sol:CounterTest
[PASS] testDouble(uint16,uint16,uint16) (runs: 256, μ: 3478271, ~: 34165)
[FAIL: assertion failed: 10750 != 9215; counterexample: calldata=0x42b5e337000000000000000000000000000000000000000000000000000000000000000100000000000000000000000000000000000000000000000000000000000000
00000000000000023fe args=[1, 9214]] testIncrement(uint16,uint16) (runs: 0, μ: 0, ~: 0)
[PASS] testSetNumber(uint16) (runs: 256, μ: 25964, ~: 29463)
Suite result: FAILED. 2 passed; 1 failed; 0 skipped; finished in 1.71s (1.75s CPU time)

Ran 1 test suite in 1.72s (1.71s CPU time): 2 tests passed, 1 failed, 0 skipped (3 total tests)

Failing tests:
Encountered 1 failing test in test/Counter.sol:CounterTest
[FAIL: assertion failed: 10750 != 9215; counterexample: calldata=0x42b5e337000000000000000000000000000000000000000000000000000000000000000100000000000000000000000000000000000000000000000000000000000000
00000000000000023fe args=[1, 9214]] testIncrement(uint16,uint16) (runs: 0, μ: 0, ~: 0)

Encountered a total of 1 failing tests, 2 tests succeeded
```

Figura 4.5: Resultado de los *tests* de funcionalidad de Foundry tras introducir un error en la función `increment`.

#### 4.1.3.2. Echidna

Para la ejecución de los *tests* se ha usado el siguiente comando:

```
echidna echidna/CounterEchidna.sol --contract CounterEchidnaTest
            --test-mode assertion --test-limit 5000
```

En el comando se indica el archivo que contiene el contrato de *testing*, el contrato de *tests*, el modo de *testing* de Echidna, y el número máximo de ejecuciones que Echidna va a realizar. Al indicar que el modo de *testing* es `assertion` Echidna ejecutará los *tests* y considerará que han fallado si la condición del `assert` se falsifica.

Al ejecutar los *tests* sin errores en el contrato, el resultado generado por la herramienta es el mostrado en la Figura 4.6.

Cuando se utiliza el modo de *testing assertion*, Echidna aplica *mutation-based fuzzing* sobre todas las funciones públicas del contrato, no únicamente aquellas que contienen aserciones (`assert()`). Como consecuencia, en el resultado generado por la herramienta aparecen más funciones además de los *tests* de interés, que son aquellos que incluyen las aserciones que permiten verificar el correcto funcionamiento del contrato.

Además de indicar que todos los *tests* se han superado, sugiriendo que el contrato se comporta correctamente, Echidna proporciona información adicional sobre el proceso de la ejecución.

Se han realizado un total de 5202 llamadas a funciones del contrato explorando diferentes combinaciones de entradas y secuencias de ejecución. El *corpus* resultante tiene un tamaño de 5, lo que representa el conjunto de entradas base utilizadas para mutar y generar nuevas secuencias. Por último, la ejecución ha cubierto 1112 instrucciones únicas.

Al ejecutar el comando se ve el proceso seguido por la herramienta y los *workers*

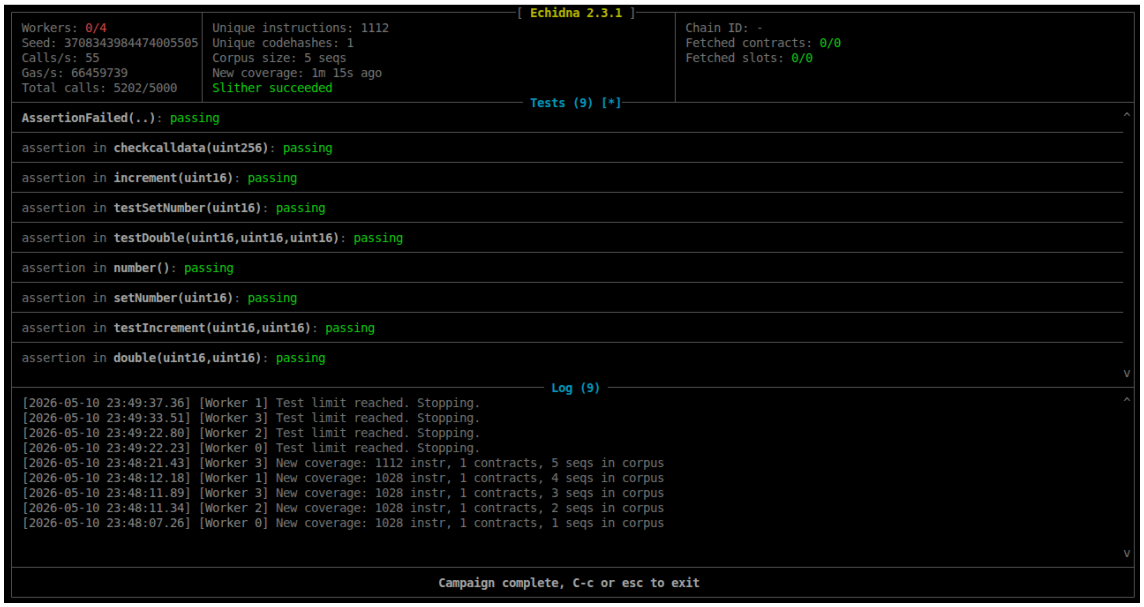



Figura 4.6: Resultado de los *tests* de funcionalidad de Echidna.

que hay activos en ese momento. Además, se muestra la semilla utilizada en la ejecución de los *tests*, esto permite reproducir el proceso de *fuzzing*.

La opción `--format text` puede añadirse al comando para obtener por terminal una salida más extensa con más información sobre la ejecución de la herramienta:

```
echidna echidna/CounterEchidna.sol --contract CounterEchidnaTest
      --test-mode assertion --test-limit 5000 --format text
```

Una vez modificada la función `increment` y ejecutados los *tests*, se obtiene la salida mostrada en la Figura 4.7. El *test* `testIncrement` ha fallado y, tras realizar el proceso de *shrinking*, se ha concluido que el fallo se puede reproducir con los parámetros `number = 0` y `times = 8`.

Debido al *shrinking* se ha podido obtener una entrada simplificada de parámetros que reproducen el fallo, lo que facilita su análisis y reproducción de forma más sencilla.

## 4.2. Evaluación del consumo de gas

El gas representa el coste computacional asociado a la ejecución de cada instrucción y cuando se ejecuta una transacción se debe pagar un precio en Ether en función del gas consumido. Por esta razón, su optimización es un aspecto clave en el desarrollo de contratos inteligentes, especialmente en contratos con bucles o cálculos repetitivos que pueden hacer que el gas consumido aumente.

El objetivo de esta evaluación es:

- Verificar que las funciones del contrato no excedan límites de gas previamente definidos.

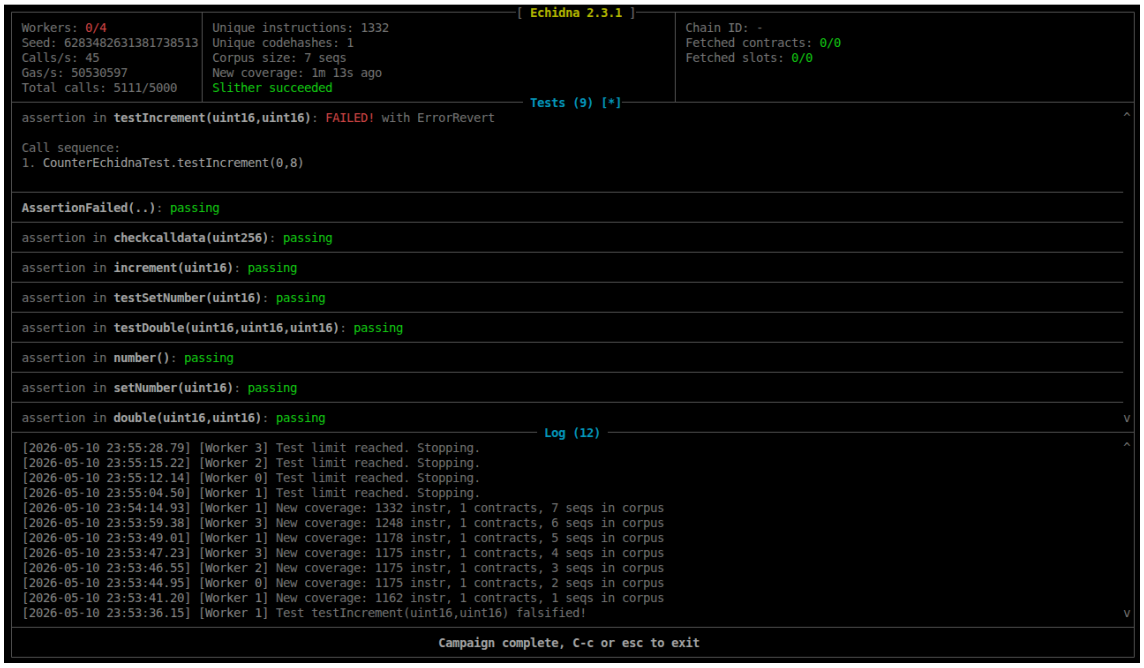



Figura 4.7: Resultado de los *tests* de funcionalidad de Echidna tras introducir un error en la función `increment`.

- Analizar la variación del consumo de gas dependiendo de la complejidad de la función.
- Evaluar si la herramienta tiene en cuenta el coste de `calldata` en la ejecución.

Para que el consumo de gas sea más manejable, se han reducido los tipos de las variables `times` de la función `increment` y `a` y `b` de la función `double` de `uint16` a `uint8`.

### 4.2.1. Diseño del conjunto de pruebas

Para poder realizar el consumo de gas de las funciones del contrato, se han desarrollado *tests* específicos que permiten medir dicho gas. Esto se hace mediante la función `gasleft`, que permite obtener la cantidad disponible de gas. Para calcular el gas consumido por una función, se hace la diferencia entre el gas restante antes y después de la llamada a dicha función. Cada *test*, excluyendo el *test* de `calldata`, tiene asociado una constante que indica el máximo gas consumible por una función, esta puede ser modificada por el usuario. En los *tests* se comprueba que el gas consumido no supere dicha constante.

Las pruebas se realizan sobre el contrato de la Figura 4.8.

En las Figuras 4.10 y 4.11 se muestran los *tests* desarrollados para Foundry. Los *tests* de consumo de gas desarrollados para Echidna son equivalentes a los usados en Foundry, aunque Echidna emplea `assert` simples a diferencia de Foundry que utiliza unos más específicos (`assertLe`). En la Figura 4.9 se puede ver un ejemplo de cómo son los *tests* de Echidna.

```
1  // File: src/Counter.sol
2  // SPDX-License-Identifier: UNLICENSED
3  pragma solidity 0.8.13;
4
5  contract CounterGas {
6      uint16 public number;
7
8
9      function setNumber(uint16 newNumber) public virtual {
10         number = newNumber;
11     }
12
13     function increment(uint8 times) public virtual {
14         require(uint256(number) + uint256(times) <= type(uint16).
               max, "Overflow");
15
16         for (uint8 i = 0; i < times; i++) {
17             number++;
18         }
19     }
20
21     function double(uint8 a, uint8 b) public virtual{
22         uint256 total = uint256(a) * uint256(b);
23         require(total <= type(uint16).max, "Overflow");
24         require(number + total <= type(uint16).max, "Overflow");
25
26         for (uint8 i = 0; i < a; i++) {
27             for (uint8 j = 0; j < b; j++) {
28                 number++;
29             }
30         }
31     }
32
33     function checkcalldata(uint256 a) public {
34         // This function is empty to measure only the cost of
               sending the data (calldata)
35     }
36 }
```

Figura 4.8: Código fuente del contrato utilizado para las pruebas de consumo de gas.

Al igual que en los *tests* para comprobar la funcionalidad del contrato, en estas pruebas se sigue usando *mutation-based fuzzing*, que permite analizar el consumo de gas bajo una amplia variedad de valores de entrada y observar cómo se comportan las funciones.

En el *test* de `calldata` no se realiza *mutation-based fuzzing*, porque el objetivo no es generar *inputs* aleatorios, sino comprobar el consumo de gas en dos casos: al llamar a la función `checkcalldata` con cero y con uno. Si la herramienta considera

el coste de `calldata`, la segunda llamada debería consumir más que la primera. Esto se debe a que, como se estudiará en el análisis de gas, el coste de `calldata` depende del contenido de los datos enviados, por lo que diferentes valores valores de entrada pueden implicar distintos consumos de gas.

```
1     function testGasIncrement(uint16 x, uint8 times) public {
2         counter.setNumber(x);
3
4         //Overflow
5         vm.assume(uint256(x) + uint256(times) <= type(uint16).max);
6
7         uint256 gasStart = gasleft();
8         counter.increment(times);
9         uint256 gasUsed = gasStart - gasleft();
10
11        // Verify gas used doesnt exceed gas limit
12        assert(gasUsed < MAX_GAS_INCREMENT);
13    }
```

Figura 4.9: Ejemplo *test* de gas de Echidna.

### 4.2.2. Análisis del consumo de gas

#### 4.2.2.1. Complejidad constante

La función `setNumber` realiza únicamente una asignación directa sobre `number`. Desde el punto de vista teórico, su complejidad es constante, $\mathcal{O}(1)$.

Esta función se evalúa mediante la prueba `testGasSetNumber`. El *fuzzer* genera un valor aleatorio para el parámetro `x` y este es utilizado como argumento en la función `setNumber`. Para que el *test* no falle, la cantidad de gas usada debe ser menor o igual a la constante `MAX_GAS_SETNUMBER` definida previamente. SI la prueba falla significa que la función consume más gas del esperado.

Aunque la complejidad de la función es constante, analizar el consumo de gas es importante, ya que pequeñas ineficiencias acumuladas pueden generar un gasto de gas significativo en el contrato.

#### 4.2.2.2. Complejidad lineal

La función `increment` ejecuta un bucle cuya cantidad de iteraciones está determinada por el parámetro `times`. Cada iteración implica una operación de incremento sobre `number`, por lo que el número total de instrucciones ejecutadas crece proporcionalmente al valor de `times`.

Teóricamente, su complejidad es lineal $\mathcal{O}(n)$, donde `n` corresponde al número de iteraciones del bucle. En consecuencia, el consumo de gas también aumentará de forma proporcional al número de iteraciones.

El *test* asociado a esta función es `testGasIncrement`, su objetivo es comprobar que el consumo de gas no pasa del límite `MAX_GAS_INCREMENT`.

Antes de la llamada a la función y ver el gas restante se introduce una restricción mediante `vm.assume` en los *tests* de Foundry y `require` en los *tests* de Echidna para evitar desbordamientos en el tipo `uint16`. De este modo las pruebas se centrarán solo en analizar el consumo de gas asociado a la función evitando que errores de ejecución interfieran en la medición.

Esto resulta relevante ya que un consumo de gas ineficiente puede provocar que las transacciones fallen y que los usuarios no puedan completar la operación.

```
1  // SPDX-License-Identifier: UNLICENSED
2  pragma solidity ^0.8.13;
3
4  import {Test} from "forge-std/Test.sol";
5  import {CounterGas} from "../src/CounterGas.sol";
6
7  contract CounterGasTest is Test {
8      CounterGas public counter;
9
10
11     function setUp() public {
12         counter = new CounterGas();
13     }
14
15     // Gas limits
16     uint256 constant MAX_GAS_SETNUMBER = 100000;
17     uint256 constant MAX_GAS_INCREMENT = 50000000;
18     uint256 constant MAX_GAS_DOUBLE = 70000000;
19
20     function testGasSetNumber(uint16 x) public {
21         uint256 gasStart = gasleft();
22         counter.setNumber(x);
23         uint256 gasUsed = gasStart - gasleft();
24
25         // Verify gas used doesnt exceed gas limit
26         assertLe(gasUsed, MAX_GAS_SETNUMBER);
27     }
28
29     function testGasIncrement(uint16 x, uint8 times) public {
30         counter.setNumber(x);
31
32         //Overflow
33         vm.assume(uint256(x) + uint256(times) <= type(uint16).max);
34
35         uint256 gasStart = gasleft();
36         counter.increment(times);
37         uint256 gasUsed = gasStart - gasleft();
38
39         // Verify gas used doesnt exceed gas limit
40         assertLe(gasUsed, MAX_GAS_INCREMENT);
41     }
```

Figura 4.10: Código de las pruebas de consumo de gas (parte 1).

```
    function testGasDouble(uint16 x, uint8 a, uint8 b) public{
        counter.setNumber(x);

        uint256 mult = uint256(a) * uint256(b);

        //Overflow
        vm.assume(mult <= type(uint16).max);
        vm.assume(uint256(x) + mult <= type(uint16).max);

        uint256 gasStart = gasleft();
        counter.double(a, b);
        uint gasUsed = gasStart - gasleft();

        assertLe(gasUsed, MAX_GAS_DOUBLE);
    }

    function testGasCallData() public {
        // warm-up
        counter.checkcalldata(0);

        uint256 gas1 = gasleft();
        counter.checkcalldata(0);
        uint256 gas2 = gasleft();

        counter.checkcalldata(1);
        uint256 gas3 = gasleft();

        uint256 gasUsed1 = gas1 - gas2;
        uint256 gasUsed2 = gas2 - gas3;

        // First call should use less gas than the second one
        assertEq(gasUsed2, gasUsed1);
    }
}
```

Figura 4.11: Código de las pruebas de consumo de gas (parte 2).

#### 4.2.2.3. Complejidad cuadrática

La función `double` ejecuta un bucle anidado cuya cantidad total de iteraciones está determinada por los parámetros `a` y `b`. Cada iteración correspondiente al bucle que tiene como límite `a` realiza `b` operaciones de incremento sobre la variable `number`, por lo que el número total de instrucciones ejecutadas crece proporcionalmente al valor de `a * b`.

La complejidad de la función `double` es $\mathcal{O}(n^2)$.

El *test* asociado a esta función es `testGasDouble`, este comprueba que el gas consumido no supere `MAX_GAS_DOUBLE`.

La implementación de funciones con esta complejidad es peligrosa, ya que permiten escenarios que matemáticamente son correctos, pero pueden exceder el límite de gas por bloque (*block gas limit*) en la red Ethereum. Este es el valor máximo de gas que puede consumirse por bloque, cuando se supera se abre la puerta a

vulnerabilidades de Denegación de Servicio (DoS) [16].

#### 4.2.2.4. Calldata

En la red Ethereum, enviar datos en los parámetros de llamada a una función tiene un coste directo. La información que el usuario envía se almacena en lo que se conoce como `calldata`. En la *Ethereum Virtual Machine*, el coste de `calldata` depende del contenido de los bytes enviados: cada byte con valor cero consume 4 unidades de gas, mientras que un byte con valor distinto de cero consume 16 unidades [37]. Por tanto, debería de haber una diferencia de consumo de gas entre ambas llamadas a `checkcalldata`.

Para analizar este comportamiento, se diseñó la prueba `testGasCallData`. El objetivo era comprobar si el contrato podía percibir este coste de entrada desde su interior utilizando la función `gasleft()`. Para ello, la prueba compara el gas consumido internamente al enviar un valor nulo (0) frente a 1.

### 4.2.3. Evaluación por herramienta

#### 4.2.3.1. Foundry

Tras ejecutar los *tests* con los valores de las constantes previamente ajustados, la herramienta muestra en la terminal el resultado de cada prueba. Cuando el resultado de un *test* es *PASS* significa que el gas consumido en la función evaluada es menor o igual que el límite definido en la constante correspondiente. Por el contrario, cuando un *test* resulta como *FAIL* significa que ha fallado, por lo que el consumo de gas de la función ha superado el límite establecido.

Para ejecutar los *tests* se pueden usar dos comandos:

- `forge test`
- `forge test --match-contract CounterGas --match-test testGasDouble --gas-report`

Con el primer comando se muestra el resultado del *test* y, en caso de fallar, indica el gas consumido por la función. En cambio, el segundo comando ofrece una salida más específica sobre el gas, ya que muestra el resultado del *test*, el gas máximo, mínimo consumido y una media del gas consumido. Este comando se debe ejecutar individualmente con cada *test* de gas.

Primero se analiza el consumo de gas con el comando más sencillo, `forge test`. Antes de ejecutar el *test* se ajustan los límites de gas para que en este primer análisis no produzca error por consumo de gas. En la Figura 4.12 se muestra el resultado de los *tests* de gas, todos han pasado por lo que no consumen más gas del definido en la constante correspondiente de cada función.

A continuación, se reducen las constantes definidas de máximo consumo de gas permitido para ver qué ocurre cuando los *tests* fallan.

Tras esta modificación se obtiene la salida mostrada en la Figura 4.13. Respecto al *test* `testGasSetNumber`, la salida indica que no ha pasado el *test* ya que se han

```
[⠊] Compiling...
[⠒] Compiling 3 files with Solc 0.8.13
[⠃] Solc 0.8.13 finished in 503.03ms
Compiler run successful!

Ran 4 tests for test/CounterGas.sol:CounterGas
[PASS] testGasCallData() (gas: 9030)
[PASS] testGasDouble(uint16,uint8,uint8) (runs: 257, μ: 1651080, ~: 45574)
[PASS] testGasIncrement(uint16,uint8) (runs: 257, μ: 67571, ~: 36491)
[PASS] testGasSetNumber(uint16) (runs: 257, μ: 24986, ~: 28471)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 832.69ms (864.96ms CPU time)
```

Figura 4.12: Resultado de los *tests* de gas de Foundry.

```
[⠊] Compiling...
[⠒] Compiling 1 files with Solc 0.8.13
[⠃] Solc 0.8.13 finished in 471.82ms
Compiler run successful!

Ran 4 tests for test/CounterGas.sol:CounterGas
[PASS] testGasCallData() (gas: 9030)
[FAIL: assertion failed: 1220083 > 100000; counterexample: calldata=0xfd8afa14000000000000000000000000000000000000000000000000000000000000021a60000000000000000000000000000000000000000000000000000000000
```

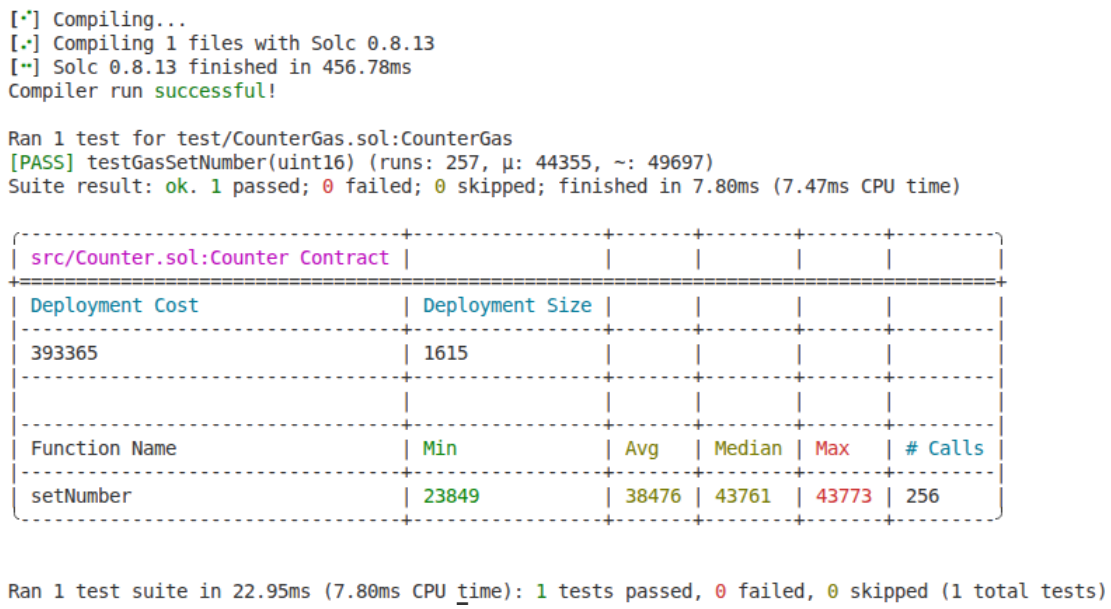



Figura 4.14: Resultado del reporte de gas en Foundry para la función `setNumber`.

que provoca que el coste de la operación de escritura dependa del estado previo del contrato.

Respecto al reporte de gas de la función `increment` mostrado en la Figura 4.15, se observa una gran diferencia entre el consumo de gas mínimo (24063) y el máximo (234221), estos resultados evidencian que el consumo de gas depende de los parámetros de entrada, lo cual es compatible con un comportamiento de complejidad lineal.

En la Figura 4.16 se muestra el reporte de gas correspondiente a la función `double`, el gas mínimo consumido son 24565 unidades, mientras que el máximo son 37574625 unidades de gas. Esta gran diferencia se debe a la dependencia del consumo de gas respecto a los parámetros de entrada `a` y `b`. Como consecuencia, el gas consumido no es constante y puede llegar a ser muy elevado en escenarios extremos.

Por último, en la Figura 4.17 se muestra el reporte de gas correspondiente a la función `checkcalldata`. La prueba falló con una levísima diferencia de 12 unidades de gas (`22486 != 22474`).

Sin embargo, hay que aclarar que esta pequeña diferencia representa el coste de `calldata`. En la segunda llamada realizada con el valor 1 como parámetro se deberían consumir 12 unidades más de gas si la herramienta tiene en cuenta `calldata`. Esto sucede porque el byte ahora es distinto de 0 por lo que su coste pasa de ser 4 a 16, 12 es la diferencia de gas entre estos dos valores.

Ejecutando el *test* con el comando que genera un reporte de gas ha hecho que `testGasCallData` falle aunque previamente con el comando `forge test` no haya fallado. Esto sucede porque ha ejecutado cada llamada a `checkcalldata` como una transacción individual, por lo que el coste de *calldata* se ha contabilizado de manera individual en cada llamada.

Según el reporte de gas, la función `checkcalldata` consume en promedio apenas

```
[⠊] Compiling...
No files changed, compilation skipped

Ran 1 test for test/CounterGas.sol:CounterGas
[PASS] testGasIncrement(uint16,uint8) (runs: 257, μ: 115186, ~: 82950)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 39.63ms (39.04ms CPU time)

| src/Counter.sol:Counter Contract |                 |       |        |        |         |
|==================================|=================|=======|========|========|=========|
| Deployment Cost                  | Deployment Size |       |        |        |         |
| 422258                           | 1749            |       |        |        |         |
|                                  |                 |       |        |        |         |
| Function Name                    | Min             | Avg   | Median | Max    | # Calls |
| increment                        | 24063           | 65188 | 32492  | 234221 | 256     |
| setNumber                        | 23871           | 40054 | 43783  | 43795  | 256     |
```

Figura 4.15: Resultado del reporte de gas en Foundry para la función `increment`.

21668 unidades de gas, pero Foundry solo está midiendo el coste de la lógica interna de la función, la cual tiene una complejidad $\mathcal{O}(1)$.

En conclusión, Foundry es una herramienta eficaz para realizar *tests* sobre el consumo de gas. Foundry incorpora comandos específicos para generar informes detallados sobre el consumo de gas facilitando el análisis del coste de cada operación del contrato. Además, la herramienta permite detectar el consumo de gas de `calldata`.

#### 4.2.3.2. Echidna

Para ejecutar los *tests* de consumo de gas de Echidna se usa el siguiente comando:

```
echidna echidna/CounterGasEchidna.sol --contract
EchidnaGasCounterTest --test-mode assertion --test-limit 5000
```

En la Figura 4.18 se muestra el resultado de la ejecución de los *tests* de gas. Se han producido un total de 5177 llamadas y se han ejecutado 1434 instrucciones únicas. Echidna también ofrece la velocidad a la que se está ejecutando el código, esta se mide en `gas/s`. Los límites de gas establecidos son altos, por lo que todos los *tests* han pasado ya que ninguna función consume más gas que el establecido.

A continuación, se reducen las constantes de gas máximo con el objetivo de analizar los resultados obtenidos con Echidna, estos son mostrados en la Figura 4.19. La ejecución de Echidna permite identificar los argumentos usados para las funciones que superan los límites de gas establecidos. En este caso, las funciones `setNumber`, `increment` y `double` consumen más gas del establecido. A diferencia de Foundry, Echidna no proporciona información específica sobre el gas consumido en las funciones que se superan el límite, sino que se limita a indicar los *tests* que se falsifican junto con la secuencia de llamadas (call secuence) y los valores de los parámetros que producen dicho fallo.

```
[⠊] Compiling...
No files changed, compilation skipped

Ran 1 test for test/CounterGas.sol:CounterGas
[PASS] testGasDouble(uint16,uint8,uint8) (runs: 257, μ: 2221756, ~: 108278)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.64s (1.63s CPU time)

╭-----------------------------------+-----------------+---------+--------+----------+---------╮
| src/Counter.sol:Counter Contract |                 |         |        |          |         |
+=================================================================================================+
| Deployment Cost                   | Deployment Size |         |        |          |         |
|-----------------------------------+-----------------+---------+--------+----------+---------|
| 422258                            | 1749            |         |        |          |         |
|-----------------------------------+-----------------+---------+--------+----------+---------|
|                                   |                 |         |        |          |         |
|-----------------------------------+-----------------+---------+--------+----------+---------|
| Function Name                     | Min             | Avg     | Median | Max      | # Calls |
|-----------------------------------+-----------------+---------+--------+----------+---------|
| double                            | 24565           | 2179308 | 59967  | 37574625 | 256     |
|-----------------------------------+-----------------+---------+--------+----------+---------|
| setNumber                         | 23871           | 39665   | 43783  | 43795    | 256     |
╰-----------------------------------+-----------------+---------+--------+----------+---------╯
```

Figura 4.16: Resultado del reporte de gas en Foundry para la función `double`.

Respecto a la función `checkcalldata`, no es posible obtener la cantidad de gas consumida directamente. No obstante, al no realizar *fuzzing* sobre este *test*, se puede modificar manualmente el `assert` para estimar los valores de `gasUsed1` y `gasUsed2`.

Tras realizar varias pruebas, se ha determinado que `checkcalldata` con parámetro 0 consume 38 unidades de gas, mientras que con parámetro 1 consume 39 unidades. Para llegar a esta conclusión, se ha ajustado la condición del `assert` del *test* `testGasCalldata`, reduciendo progresivamente hasta llegar al mínimo valor que no provoque un fallo en el *test*.

Las llamadas a la función `checkcalldata` han consumido poco porque ambas se han ejecutado en la misma transacción, Echidna no las ha considerado como transacciones individuales a diferencia de Foundry.

Hay una diferencia de una unidad de gas porque al llamar a `checkcalldata` con cero como parámetro, se ejecuta un `push0` que consume 2 unidades de gas, en cambio, al llamar a la función con cualquier otro parámetro distinto de cero se ejecuta un `push` que consume 3 unidades de gas.

Esto indica que Echidna no está teniendo en cuenta el coste real de `calldata`, mientras que la EVM sí se tiene en cuenta. La razón es porque Echidna ejecuta las funciones como llamadas internas y no simula una transacción completa.

En conclusión, Echidna es útil para comprobar si las funciones de un contrato consumen más gas que un máximo definido por el usuario, pero no refleja el coste total de una transacción en la red Ethereum.

## 4.3. Evaluación de vulnerabilidades: Reentrada

Esta seccion se centra en una de las vulnerabilidades más conocidas y críticas dentro del ecosistema de *blockchain*: reentrada. La vulnerabilidad de reentrada es un

```
[⠒] Compiling...
No files changed, compilation skipped

Ran 1 test for test/CounterGas.sol:CounterGas
[FAIL: assertion failed: 22486 != 22474] testGasCallData() (gas: 75739)
Suite result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 446.57µs (124.71µs CPU time)

╭-----------------------------------+-----------------+-------+--------+-------+---------╮
| src/Counter.sol:Counter Contract |                 |       |        |       |         |
+=============================================================================================+
| Deployment Cost                   | Deployment Size |       |        |       |         |
|-----------------------------------+-----------------+-------+--------+-------+---------|
| 393353                            | 1615            |       |        |       |         |
|-----------------------------------+-----------------+-------+--------+-------+---------|
|                                   |                 |       |        |       |         |
|-----------------------------------+-----------------+-------+--------+-------+---------|
| Function Name                     | Min             | Avg   | Median | Max   | # Calls |
|-----------------------------------+-----------------+-------+--------+-------+---------|
| checkcalldata                     | 21664           | 21668 | 21664  | 21676 | 3       |
╰-----------------------------------+-----------------+-------+--------+-------+---------╯
```

Figura 4.17: Resultado del reporte de gas en Foundry para la función `checkcalldata`.

fallo de lógica, especifícamente en la sincronización del estado interno de un *smart contract*.

Para poder analizar esta vulnerabilidad, se ha creado un contrato vulnerable, un contrato que actuará como atacante y una serie de *tests* para detectar la vulnerabilidad.

A diferencia de las evaluaciones realizadas anteriormente, en este caso no se emplea *mutation-based fuzzing*, sino que se diseñan escenarios específicos con el objetivo de reproducir manualmente la vulnerabilidad y evaluar la capacidad de las herramientas para detectarla.

El objetivo de esta evaluación es:

- Recrear la vulnerabilidad de reentrada.
- Comprobar si las herramientas son capaces de detectar la vulnerabilidad mediante el uso de *tests*.

### 4.3.1. Contexto y concepto de la vulnerabilidad

En la *Ethereum Virtual Machine*, los *Smart Contracts* tienen la capacidad de invocar y ceder el flujo de ejecucion a otros contratos externos, habitualmente mediante transferencias o llamadas de bajo nivel como `.call{value: X}()`. En resumen, esta vulnerabilidad se da cuando tenemos un contrato **A** que hace una llamada al contrato **B** antes de haber actualizado su propio estado interno. Hasta que el contrato **A** vuelva a tomar el control, el contrato **B** puede aprovechar esta "pausa" y realizar una llamada reentrante a **A** para aprovechar su estado interno inconsistente.

Como se observa en la Figura 4.20, la función `withdraw` primero realiza una llamada externa al usuario mediante `msg.sender.call` y después actualiza el estado interno del contrato. Este orden es incorrecto y permite la vulnerabilidad de reentrada.

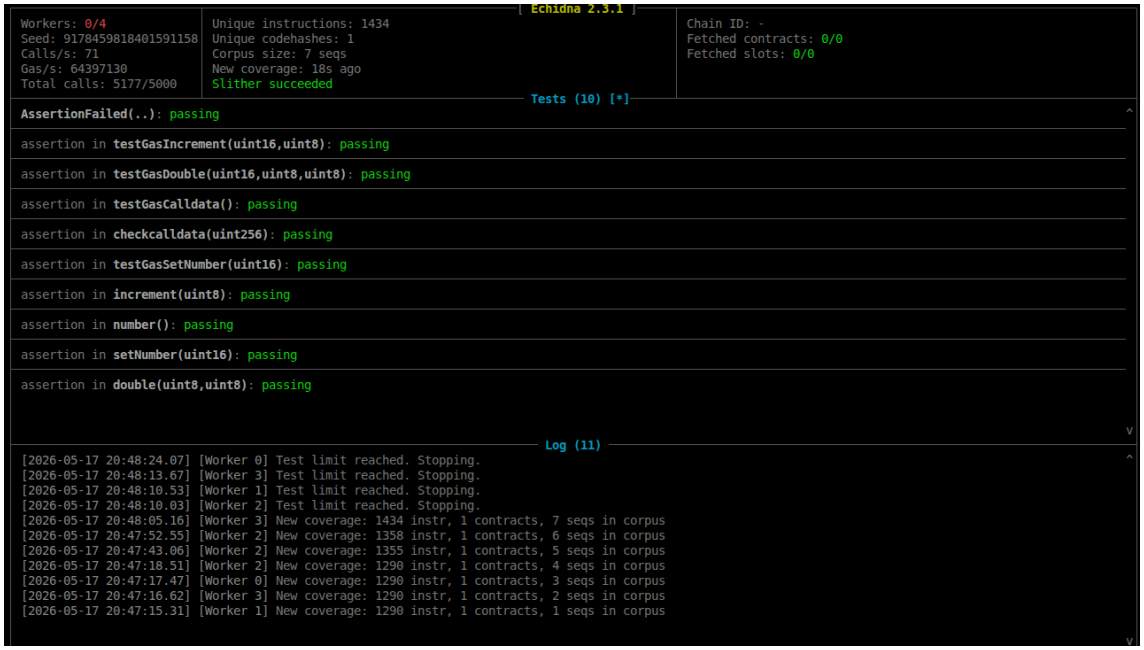



Figura 4.18: Resultado de los *tests* de gas en Echidna.

Cuando el contrato atacante mostrado en la Figura 4.21 llama a `withdraw`, el contrato vulnerable envía Ether antes de actualizar su almacenamiento. Esta tansferencia activa automáticamente la función `receive` del atacante, que vuelve a llamar a `withdraw`.

En el momento en el que el atacante hace la segunda llamada a `withdraw`, su saldo todavía no ha disminuido. Por eso, cada vez que se reentra en la función, la condición `require(balance >0` sigue cumpliéndose y el contrato permite retirar fondos otra vez.

La ejecución continúa hasta que el contrato se queda sin fondos o hasta que se agote el gas (*Out-Of-Gas*).

Esta vulnerabilidad ha causado algunos de los mayores hackeos en la historia de las finanzas descentralizadas (DeFi), como el caso de *The DAO* ya mencionado anteriormente.

Podría parecer un error del pasado, pero en julio de 2023, un protocolo llamado *Curve Finance*, uno de los mayores creadores de mercado automatizado, sufrió un ataque similar que resultó en el robo de unos 70m de dolares [15]. Lo curioso es que no fue un fallo de programacion del codigo escrito, sino por un error del propio compilador del lenguaje Vyper [36].

Estos casos nos demuestran que un código que a simple vista parece correcto puede tener fallos de lógica muy graves. Por eso, analizar los contratos con herramientas automatizadas de *fuzzing* es totalmente necesario antes de publicar cualquier contrato en la red ETH, ya que son inmutables.

Para evitar estas vulnerabilidades, los estándares de seguridad *Web3* exigen seguir el patrón *Checks-Effects-Interactions*. Una regla básica es el orden en el que se debe desarrollar el código de una función:

1. ***Checks***: Verificar las condiciones necesarias. Por ejemplo, validar mediante un

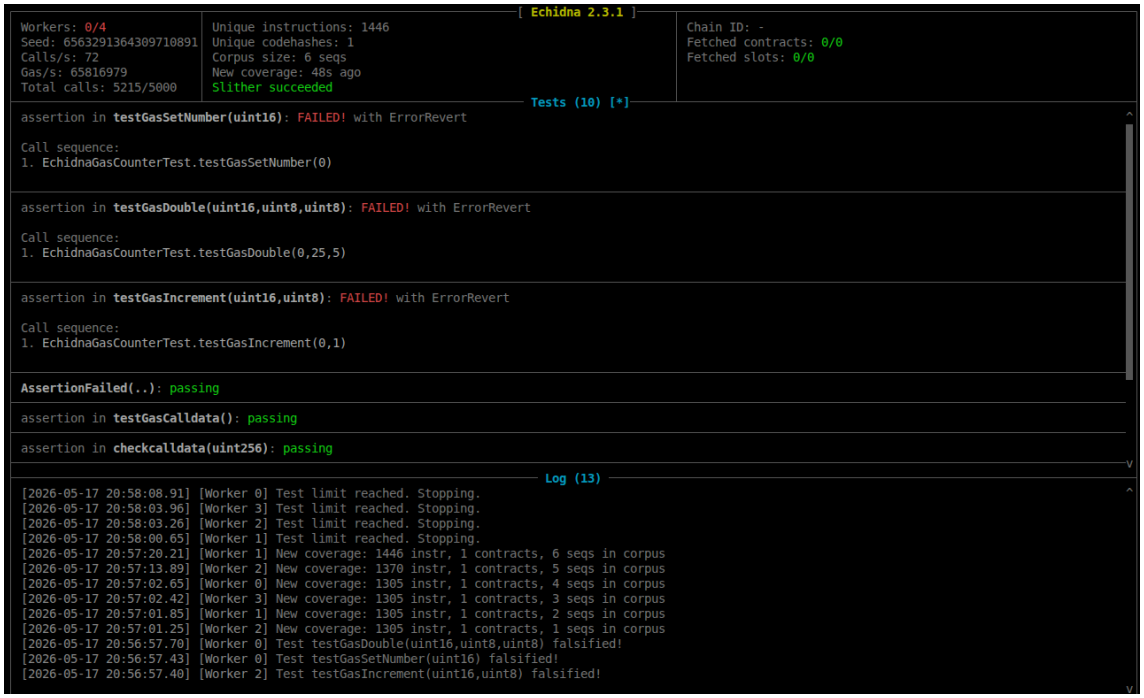



Figura 4.19: Resultado de los *tests* de gas en Echidna tras reducir las constantes de gas máximo consumido.

`require` que el usuario disponga de saldo disponible.

2. ***Effects***: Actualizar las variables de estado interno del contrato.
3. ***Interactions***: Finalmente, realizar la transacción de *Ether* o la llamada al contrato externo.

El contrato de la Figura 4.20, se salta esta regla a propósito. Como se puede apreciar en el código a continuación, ejecuta la *Interaction* (línea 17) antes de aplicar el *Effect* (línea 20). Creando una *Race Condition*. En los siguientes experimentos, comprobaremos en la práctica cómo las herramientas de *mutation-based fuzzing* logran explotar este desajuste.

### 4.3.2. Evaluación con Foundry

Para realizar esta evaluación, se ha preparado un entorno de pruebas que simula un escenario realista. El código fuente del contrato vulnerable se expone en la Figura 4.20, la lógica del contrato atacante y los *tests* se detallan en la Figura 4.21 y la Figura 4.22, respectivamente.

Durante la configuración inicial (función `setUp()`), un usuario normal deposita 10 Ether en el contrato `VulnerableBank`. Seguidamente, se introduce un contrato atacante (`MaliciousAttacker`) financiado con 1 Ether de saldo inicial. Este contrato incorpora una función especial de recepción (`receive() external payable`), programada a propósito para realizar el ataque.

Para darle rigor a la prueba unitaria (`test_bank_is_secure()`), en lugar de comprobar si el atacante consigue robar el dinero, la atención se centra directamente

en el saldo del banco. En concreto, se verifica si el contrato es capaz de mantener a salvo los depósitos de los clientes tras sufrir el ataque, asegurando mediante un `assert` que su balance final siga siendo de 10 Ether.

```
1  // SPDX-License-Identifier: UNLICENSED
2  pragma solidity 0.8.13;
3
4  // Vulnerable Contract
5  contract VulnerableBank {
6      mapping(address => uint256) public balances;
7
8      function deposit() public payable {
9          balances[msg.sender] += msg.value;
10     }
11
12     // Vulnerable function: sends Ether before updating balance
13     function withdraw() public {
14         uint256 balance = balances[msg.sender];
15         require(balance > 0, "Insufficient balance");
16
17         (bool success, ) = msg.sender.call{value: balance}("");
18         require(success, "Transfer failed");
19
20         balances[msg.sender] = 0;
21     }
22 }
```

Figura 4.20: Código fuente del contrato `VulnerableBank.sol`.

Al ejecutar las pruebas, Foundry comprueba la aserción y detecta el fallo de seguridad al instante. Como muestra la Figura 4.23, el *test* falla arrojando un error en rojo porque el balance del banco cae a 0 en lugar de conservar los 10 Ether esperados (que en la terminal se muestran en su unidad mínima, *wei*, como `10000000000000000000`).

Además, la traza de llamadas (*Call Trace*) de esa misma figura permite ver paso a paso cómo funciona el ataque. Si nos fijamos en el árbol de ejecución que se va desplazando hacia la derecha, vemos la trampa en acción: Primero, el atacante pide retirar su Ether original (`10000000000000000000` wei). Justo cuando el banco le envía esos fondos usando una llamada `call`, la función `receive()` del atacante toma el control. En lugar de simplemente aceptar el dinero, aprovecha esa pausa para volver a llamar a `withdraw()` por segunda vez.

Como el banco todavía no ha tenido tiempo de actualizar el saldo en su memoria, le vuelve a dar otro Ether. Este patrón en forma recursivo y consumiendo gas en cada salto, como indican los números entre corchetes: [92709], [85026], [84238]... hasta que consigue vaciar por completo los fondos del contrato.

### 4.3.3. Evaluación con Echidna

Para poder realizar la evaluación de la presencia de reentrada con Echidna, se ha usado el contrato vulnerable usado para Foundry mostrado en la Figura 4.20. El

```
1  // SPDX-License-Identifier: UNLICENSED
2  pragma solidity 0.8.13;
3
4  import "../src/Reentrancy.sol";
5
6  // Classic Attacker Contract
7  contract MaliciousAttacker {
8      VulnerableBank public bank;
9
10     constructor(address _target) {
11         bank = VulnerableBank(_target);
12     }
13
14     // The attacker injects legitimate funds to pass the require(
           balance > 0)
15     function deposit() external payable {
16         bank.deposit{value: msg.value}();
17     }
18
19     // Attack trigger
20     function attack() external {
21         bank.withdraw();
22     }
23
24     receive() external payable {
25         // As long as the bank has funds, we continue the
               reentrancy
26         if (address(bank).balance >= msg.value) {
27             bank.withdraw();
28         }
29     }
30 }
```

Figura 4.21: Código fuente del contrato `MaliciousAttacker`.

contrato que simulará el atacante y el *test* que verificará si hay reentrada o no en el contrato se muestran en la Figura 4.24 y Figura 4.25 respectivamente.

Al desplegar los contratos, se ejecutan sus correspondientes constructores. Durante la ejecución del constructor de la Figura 4.25 se depositan 10 Ether en el banco vulnerable (`target`) siendo este el estado inicial.

En el contrato de la Figura 4.24 cuando se ejecute la función `attack`, el atacante depositará 1 Ether en el banco mediante la función `deposit` del banco y después procederá a retirarlo mediante la función `withdraw`. Como esta función realiza una llamada externa al usuario antes de actualizar su estado, hará que se ejecute la función `receive` de `MaliciousAttacker` que, mientras el banco siga teniendo fondos, volverá a llamar a `withdraw`.

La propiedad definida para comprobar si hay reentrada o no se define en la Figura 4.25. La función `echidna_test_vault_never_drained` comprueba que el banco nunca tenga depositados menos de 10 Ether. Si no hubiera reentrada, el banco tendría 10 Ether, el atacante depositaría 1 Ether y lo retiraría por lo que el balance del banco seguiría en 10 Ether y el *test* no fallaría.

```
1  import "forge-std/Test.sol";
2
3  contract ReentrancyTest is Test {
4      VulnerableBank bank;
5      MaliciousAttacker attacker;
6
7      function setUp() public {
8          bank = new VulnerableBank();
9          attacker = new MaliciousAttacker(address(bank));
10
11         // 1. Simulate real scenario: An honest user deposits 10
               ether into the bank
12         vm.deal(address(0x1), 10 ether);
13         vm.prank(address(0x1));
14         bank.deposit{value: 10 ether}();
15     }
16
17     function test_bank_is_secure() public {
18         // 2. The attacker gets funded with 1 own ether and
               deposits it
19         vm.deal(address(attacker), 1 ether);
20         attacker.deposit{value: 1 ether}();
21
22         // 3. The attacker triggers the asynchronous loop by
               calling a normal withdraw
23         attacker.attack();
24
25         // 4. SECURITY TEST (The bank MUST be secure)
26         // The bank should keep the 10 ether of the honest user
               intact.
27         // Since the attacker drains it, this line will FAIL,
               showing the red [FAIL]
28         assertEq(
29             address(bank).balance,
30             10 ether,
31             "Peligro de Vulnerabilidad: El banco ha perdido el
                   dinero de los usuarios"
32         );
33     }
34 }
```

Figura 4.22: Código fuente del entorno de pruebas `ReentrancyTest.sol`.

A partir del estado inicial, Echidna comienza a generar secuencias de llamadas de forma aleatoria sobre las funciones públicas del contrato, después de cada secuencia se evalúa la propiedad definida en el *test*.

Para ejecutar las pruebas se ha utilizado el siguiente comando:

```
echidna echidna/ReentrancyEchidnaTest.sol --contract
 ReentrancyEchidnaTest --config echidna/config.yaml
```

El archivo `config.yaml` permite definir condiciones iniciales del entorno de ejecución, su contenido se muestra en la Figura 4.26. El parámetro `testMode:`

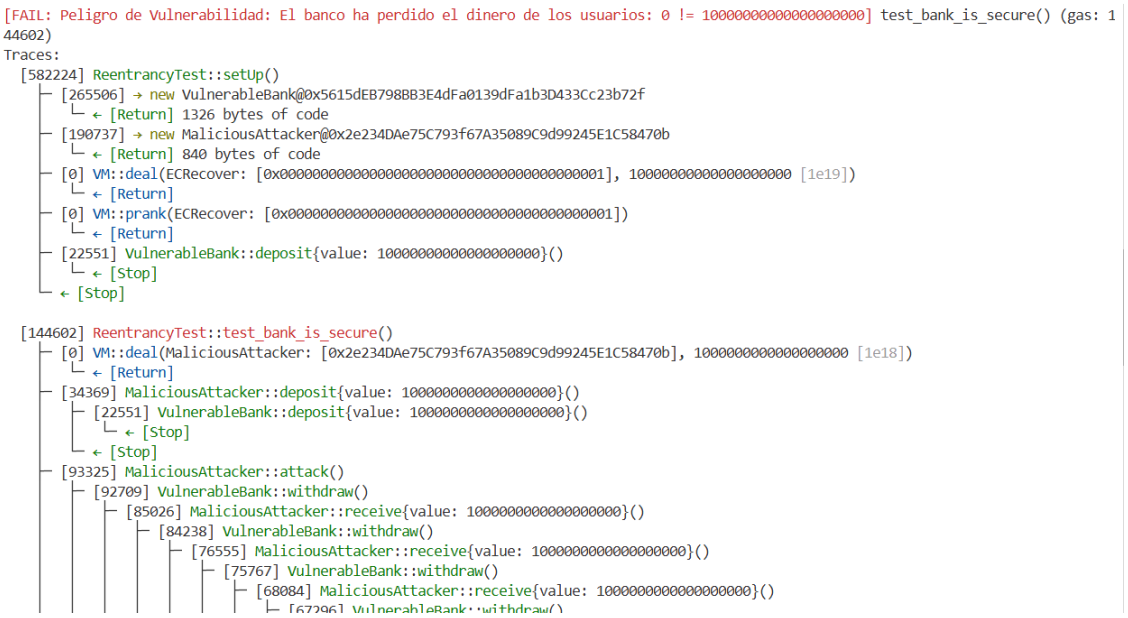



Figura 4.23: Resultado de la evaluación de reentrada y traza de llamadas utilizando Foundry.

```
1  import "../contracts/VulnerableBank.sol";
2
3  contract MaliciousAttacker {
4      VulnerableBank public target;
5
6      constructor(VulnerableBank _target) {
7          target = _target;
8      }
9
10     receive() external payable {
11         if (address(target).balance >= msg.value) {
12             target.withdraw();
13         }
14     }
15
16     function attack() public payable {
17
18         uint256 amount = 1 ether;
19         if(address(this).balance >= amount) {
20             target.deposit{value: amount}();
21             target.withdraw();
22         }
23     }
24 }
```

Figura 4.24: Código fuente de `MaliciousAttacker.sol` para Echidna.

''`property`'' indica que Echidna debe operar en modo basado en propiedades, generará secuencias de llamadas con el objetivo de encontrar un estado en el que las función que empiezan por `echidna_` devuelvan `false`. El parámetro `corpusDir`:

```
contract ReentrancyEchidnaTest is MaliciousAttacker {
    constructor() MaliciousAttacker(new VulnerableBank()) payable {
        target.deposit{value: 10 ether}();
    }

    function echidna_test_vault_never_drained() public view returns
        (bool) {
        return address(target).balance >= 10 ether;
    }
}
```

Figura 4.25: Código fuente de los *tests* desarrollados para Echidna.

''corpus'' especifica el directorio donde Echidna almacenará las secuencais de entradas (*corpus*) que considera interesantes. Por último, el parámetro `balanceContract` asigna fondos al contrato bajo *test*, en este caso `ReentrancyEchidnaTest`, expresados en wei (unidad base de Ethereum) correspondiendo en este caso a 20 Ether. A partir de estos fondos, en el constructor de este contrato se realiza el depósito de 10 Ether en el contrato vulnerable.

```
testMode: "property"
corpusDir: "corpus"
balanceContract: 20000000000000000000
```

Figura 4.26: Contenido `config.yaml`.

El resultado de la ejecución mostrado en la Figura 4.27 indica que la propiedad ha fallado y la secuencia de llamadas que ha ejecutado y han provocado el error ha sido llamando a `attack`. En las trazas se puede observar la secuencia explicada anteriormente, la función definida como `unknown method` es la función `receive` de `MaliciousAttacker`.

En definitiva, este *test* comprueba que Echidna es capaz de identificar vulnerabilidades de reentrada de forma eficiente.

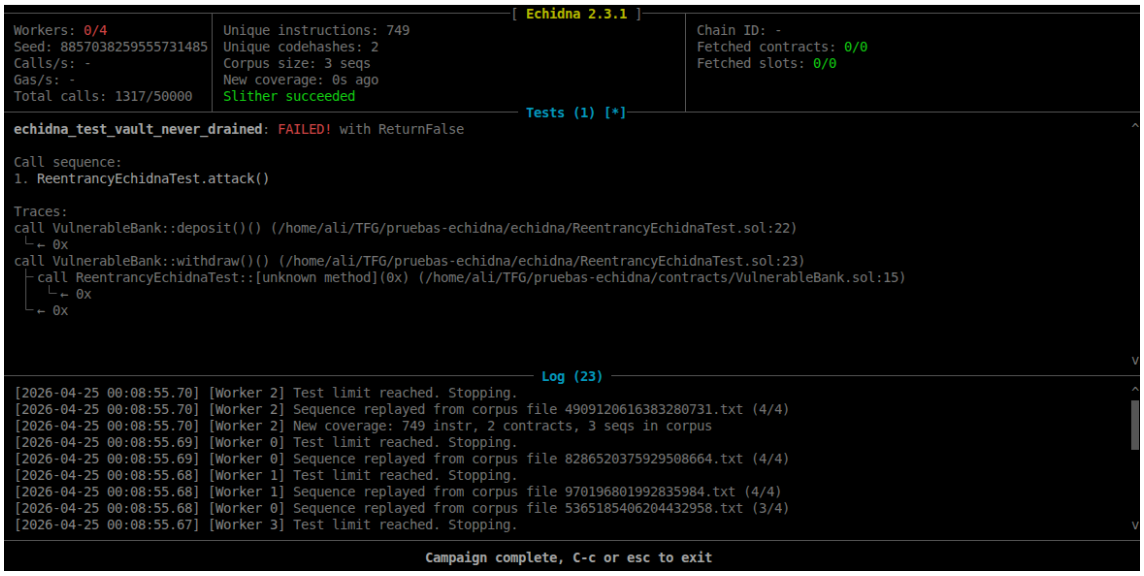



Figura 4.27: Resultado de la evaluación de Reentrancy y traza de llamadas utilizando Echidna.

Capítulo 5

# Análisis Comparativo de las herramientas

En este capítulo se realiza un análisis comparativo de las herramientas. Para cada herramienta se explica la instalación de esta y se describe su facilidad de uso. A continuación, se estudian las ventajas y desventajas de las herramientas según los resultados obtenidos en los experimentos realizados. Por último, se realiza una comparación de las principales caracteríscas entre ambas herramientas

## 5.1. Foundry

### 5.1.1. Instalación de la herramienta

La instalación de Foundry puede realizarse mediante el siguiente comando [1]:

```
curl -L https://foundry.paradigm.xyz | bash
foundryup
```

El primer comando descarga Foundry, mientras que el segundo instala y actualiza los componentes necesarios de la herramienta en el sistema. Para comprobar que la herramienta se ha instalado correctamente se puede verificar mediante el comando `forge -version`. Una vez se han ejecutado ambos comandos, se puede utilizar Foundry.

Para crear la estructura básica de un proyecto se puede utilizar el comando `foundry init`, con el que genera automáticamente dicha estructura y se podrá comenzar a desarrollar contratos y *tests*.

En general el proceso de instalación es rápido y sencillo, ya que no requiere configuración adicional ni la instalación de dependencias complejas.

### 5.1.2. Facilidad de uso

Foundry es una herramienta sencilla de instalar y, al no disponer de una gran variedad de modos de ejecución, la herramienta resulta manejable y fácil de comprender. El comando básico (`forge test`) permite ejecutar las pruebas necesarias sobre los contratos sin necesidad de realizar configuraciones adicionales. Foundry detecta las funciones de prueba por su nombre, considerando como *tests* las funciones

cuyo nombre comienza por `test`. En caso de necesitar funcionalidades más específicas, como la generación de reportes de gas, Foundry incorpora comandos adicionales que también resultan fáciles de entender y utilizar.

El desarrollo de los *tests* también es intuitivo ya que se realizan directamente en Solidity. La organización del proyecto resulta clara, lo que permite una gestión sencilla de los contratos y *tests* a lo largo del desarrollo.

La salida proporcionada por Foundry es bastante minimalista, lo que facilita su lectura y permite identificar rápidamente la información relevante sobre la ejecución de los *tests*. Incluso en funcionalidades más específicas, como los reportes de gas, la información se presenta de forma clara y organizada.

En general, Foundry presenta una curva de aprendizaje baja y permite comenzar a desarrollar y ejecutar *tests* rápidamente sin necesidad de comprender conceptos complejos ni realizar configuraciones avanzadas.

### 5.1.3. Ventajas

Los resultados de la ejecución de los *tests* con Foundry indican si los *tests* han pasado o no, y además proporcionan información adicional como cuántas veces se ha ejecutado, la media de gas consumido y la mediana. Cuando un *test* falla, la herramienta indica la entrada y la condición del `assert` que ha provocado el fallo del *test*. Esto es una ventaja ya que en *tests* como los de consumo de gas, si la función estudiada consume más del máximo definido, permite ver el gas consumido.

Respecto al consumo de gas, Foundry es especialmente útil porque tiene un comando específico para realizar reportes de gas. Este comando separa las funciones llamadas en el *test* y permite saber el mínimo, la media, la mediana y el máximo gas consumido por cada una de las funciones. Además, en la prueba realizada la diferencia de gas entre la llamada a `calldata` con 0 y con 1 es de doce unidades de gas, lo que indica que Foundry sí tiene en cuenta el gasto de `calldata`.

El desarrollo de los contratos y el *test* para recrear e identificar la vulnerabilidad de reentrada ha sido sencillo. En Foundry es fácil dirigir el flujo de ejecución de la herramienta por lo que recrear diferentes vulnerabilidades resulta más sencillo que con otras herramientas. Tras la ejecución del *test* de reentrada, se concluyó que, mediante los *tests* correctos, Foundry es capaz de detectar este tipo de vulnerabilidad.

Una de las principales ventajas de Foundry es que las trazas que genera son muy completas, ya que muestran la secuencia exacta de llamadas, los valores utilizados y el flujo de ejecución que conduce al fallo, facilitando tanto la depuración como la comprensión de la vulnerabilidad detectada.

Foundry es una herramienta que proporciona información suficiente sobre la ejecución de los *tests* y permite realizar reportes de gas de *tests* en los que se estudia por separado el gas consumido de cada función llamada en dicho *test*.

### 5.1.4. Desventajas

Foundry dispone de menos modos de ejecución en comparación con otras herramientas existentes, como Echidna, lo que reduce la variabilidad de tipos de *tests* que se pueden realizar y, en consecuencia, la diversidad de resultados obtenidos mediante

su ejecución.

Cuando se ejecutan *tests* en Foundry los resultados obtenidos por la herramienta son bastante minimalistas, ya que se muestra directamente el final de la ejecución en lugar de una salida que se actualiza de forma continua durante la campaña de *fuzzing* avanza. Además, la información adicional proporcionada es más limitada en comparación con otras herramientas por lo que en caso de necesitar un análisis más detallado, puede ser necesario recurrir a estas.

De igual manera, en los *tests* ejecutados con Foundry se ha detectado como desventaja que la herramienta no realiza *shrinking* en caso de que un *test* falle. Esto implica que, en la salida de la herramienta, los valores de los parámetros no son los mínimos valores para poder reproducir el error. El hecho de no disponer de este proceso puede dificultar el análisis del error en ciertos casos.

## 5.2. Echidna

### 5.2.1. Instalación de la herramienta

En el caso de instalar Echidna en un sistema Linux, puede ser necesario configurar un entorno de Python de la siguiente manera [31]:

```
curl -fsSL https://pyenv.run | bash pyenv install 3.13.7
```

Primero, se debe instalar Slither mediante el siguiente comando:

```
pip3 install slither-analyzer
```

Una vez configurado el entorno, es necesario instalar la última versión de Echidna disponible en su repositorio[1] y ejecutar los siguientes comandos:

```
tar -xvzf echidna-2.3.0-x86_64-linux.tar.gz

sudo mv echidna /usr/local/bin/
```

Para comprobar que la herramienta se ha instalado correctamente se puede verificar mediante el comando `echidna -version`.

### 5.2.2. Facilidad de uso

Echidna es una herramienta más compleja de utilizar en comparación con Foundry. Tiene distintos modos de *testing*, por lo que es necesario saber qué tipo de *tests* se deben desarrollar para cada uno de los modos.

La ejecución de la herramienta no es tan directa como Foundry. Si no se quiere realizar pruebas con el modo predeterminado (`property`), es necesario indicarlo en el comando o en el archivo de configuración. Aunque existen varios campos que no requieren configuración porque tienen un valor predeterminado, como el `test-limit`, resulta útil configurarlos porque el tiempo de ejecución de los *tests* puede ser alto.

[1] https://github.com/crytic/echidna/releases

La salida proporcionada por Echidna es bastante completa y organizada, aunque incluye información adicional que es necesaria de entender para poder interpretarla correctamente. Sin embargo, una vez se comprende el funcionamiento de la herramienta, esta es muy potente por todas las variedades que ofrece.

En general, Echidna presenta una curva de aprendizaje más elevada, pero ofrece mayor flexibilidad e información útil.

### 5.2.3. Ventajas

Echidna destaca por tener varios modos de ejecución, lo que facilita desarrollar distintos tipos de *tests* dependiendo del comportamiento que se quiera analizar. El modo de *testing* predeterminado está basado en propiedades, por lo que los *tests* creados tienen que definir invariantes del código. Este tipo de *tests* resultan útiles para detectar vulnerabilidades en el código, como se ha hecho en el *test* de reentrada. En cambio, cuando se quieren realizar *tests* específicos del comportamiento de un contrato, si se desea probar que funciones específicas se comporten correctamente, es necesario realizar otros tipos de *tests*, como en los basados aserciones, ya que utilizando el modo predeterminado no se podría realizar este tipo de pruebas.

La salida proporcionada por Echidna es bastante completa, a medida que avanza la campaña de *fuzzing* se actualizan los datos de la salida constantemente. Al terminar la ejecución, se puede apreciar el resultado final de los *tests*. Cuando un *test* falla, Echidna realiza *shrinking* de forma automática, reduciendo el caso que ha hecho fallar el *test* al mínimo necesario para reproducir este error. Además, se puede configurar cuántos intentos de *shrinking* tiene la herramienta a través del comando con el que se ejecutan los *tests* o en el archivo de configuración de la herramienta.

Durante la campaña de *fuzzing*, Echidna proporciona métricas que otras herramientas no suelen mostrar, como el gas consumido por segundo, las llamadas realizadas por segundo, cuando se produce una nueva cobertura o cuanto tiempo ha pasado desde que se amplió la cobertura.

Respecto a la vulnerabilidad de reentrada, se ha conseguido desarrollar unos contratos que recrean la vulnerabilidad y se ha conseguido detectarla Además, al ejecutar *tests* en el modo predeterminado de la herramienta (`property`), Echidna muestra la secuencia de llamadas realizadas que han hecho que la propiedad se falsifique.

En conclusión, Echidna es una herramienta versátil que permite desarrollar distintos tipos de pruebas siempre que los *tests* se adapten al modo de ejecución. Además, aporta bastante información sobre la campaña de *fuzzing* y los resultados obtenidos.

### 5.2.4. Desventajas

Aunque la salida de Echidna proporciona bastante información, si durante la ejecución de *tests* uno falla, no indica cuál es la condición del `assert` cuando este falla.

En las pruebas relacionadas con la funcionalidad del contrato no es un gran problema porque Echidna sí proporciona los valores de los parámetros con los que

se ha llamado al *test* y el usuario puede comprobar por qué el `assert` ha fallado. En cambio, para los *tests* relacionados con el consumo de gas, no se puede saber por cuánto gas ha excedido el máximo de gas definido. Solo se podrá saber si la función ha consumido más o menos gas que el máximo. Por esta razón, para el *test* diseñado para comprobar si Echidna tiene en cuenta el gasto de `calldata`, ha sido necesario realizar un `assert` para cada llamada e ir disminuyendo el límite de gas, hasta encontrar la mínima cantidad de gas necesaria para la ejecución de la transacción.

Con la ejecución del *test* específico de `calldata`, se llegó a la conclusión de que Echidna no tiene en cuenta el gasto de `calldata`, tal y como se ha explicado previamente en la evaluación de las pruebas de consumo de gas. En la prueba realizada se llamó a una función vacía primero con 0 como parámetro y luego con 1. La diferencia de gas consumido entre ambas llamadas fue de una unidad de gas en lugar de doce. El hecho de que la herramienta no tenga en cuenta este coste hace que el usuario no pueda conocer con precisión el coste real de la ejecución en Ethereum.

Respecto a la vulnerabilidad de reentrada, ha sido complicado recrearla ya que cuando se usa el modo de *testing* predeterminado en Echidna es difícil crear el flujo de ejecución específico que recreará esta vulnerabilidad. En este modo, Echidna llama a varias funciones públicas del contrato con parámetros aleatorios y comprueba si la propiedad se sigue cumpliendo. El orden de las llamadas de la secuencia va cambiando dependiendo de como haya sido la generación interna de la secuencia a través de *mutation-based fuzzing*.

Aunque Echidna es una herramienta relativamente sencilla de utilizar, otras herramientas como Foundry son más flexibles y ofrecen información más útil sobre los *tests* realizados.

## 5.3. Comparativa

Tras el análisis de cada herramienta por separado, en la Tabla 5.1 se muestra una comparación de diferentes características entre Foundry y Echidna.

| **Característica** | **Foundry** | **Echidna** |
|---|---|---|
| Facilidad de uso | Alta | Media |
| Modos de testing | Limitado | Variado |
| Salida ejecución | Minimalista, final de la ejecución | Actualización continua y detallada |
| Información fallos | Entrada y condición `assert` | Entrada |
| Shrinking | Solo invariantes | Sí |
| Análisis de gas | Completo | Limitado |
| Calldata gas | Lo considera | No lo tiene en cuenta |
| Reproducción reentrada | Sencilla | Compleja |

Tabla 5.1: Comparación de herramientas de testing.

Foundry es una herramienta sencilla y fácil de utilizar. El resultado de los *tests* de funcionalidad es completo, aunque podría mejorarse si incorporase un proceso

de *shrinking*, por lo que en este aspecto otras herramientas como Echidna pueden resultar más adecuadas. En los *tests* de consumo de gas, Foundry proporciona una información más detallada que otras herramientas ya que con un comando sencillo permite conocer el gas consumido por la función estudiada cuando un *test* falla. Además, los reportes de gas realizados por la herramienta son bastante completos e incluyen el gasto de `calldata`.

Respecto a la vulnerabilidad de reentrada, su reproducción ha sido relativamente sencilla, ya que el flujo de ejecución se puede controlar de forma más directa, lo que ha permitido tanto recrear como detectar la vulnerabilidad.

Echidna es una herramienta más compleja, pero dispone de distintos modos de ejecución, lo que la convierte en una opción más flexible. En los *tests* de funcionalidad del contrato, la salida proporcionada es bastante completa y, cuando un *test* falla, realiza *shrinking* mostrando la entrada mínima necesaria para reproducir el error. En cambio, para los *tests* de consumo de gas resulta menos adecuada, ya que únicamente indica si el consumo supera o no el máximo definido por el usuario, sin aportar información detallada sobre el gas consumido. Además, no tiene en cuenta el gasto de `calldata`.

La reproducción de la vulnerabilidad de reentrada ha sido más compleja, debido a que el flujo de ejecución está basado en *fuzzing* aleatorio. Aunque la herramienta ha permitido detectarla, la secuencia de llamadas proporcionada en caso de fallo no es tan detallada como la ofrecida por Foundry.

Capítulo 6

# Conclusiones y Trabajo Futuro

En este capítulo se recogen las conclusiones obtenidas a lo largo del trabajo, y se presentan posibles líneas de trabajo futuro.

## 6.1. Conclusiones

El objetivo principal del trabajo ha sido estudiar el funcionamiento de Foundry y Echidna mediante el análisis de su estructura interna y la realización de diferentes pruebas para evaluar las capacidades de las herramientas.

Se ha evaluado su capacidad para detectar errores de diseño en los contratos que afectaban a su correcta funcionalidad, así como la medición del consumo de gas de las funciones. Para ello, se ha utilizado la función `gasleft` y, en el caso de Foundry, un comando específico para realizar un reporte de gas sobre funciones concretas. Además, se ha analizado la capacidad de las herramientas de recrear la vulnerabilidad de reentrada y, mediante la creación manual de *tests* específicos, la capacidad de detección de esta vulnerabilidad.

Tras realizar los diferentes casos de uso, se ha observado que cada herramienta presenta ventajas diferentes. Foundry destaca por su facilidad de uso, por los reportes de gas que la herramienta puede generar sobre cada función del contrato y, además por detectar el coste asociado a `calldata`. En cambio, Echidna destaca por la variedad de modos de *testing* que ofrece y por el proceso de *shrinking* que realiza cuando un *test* falla.

Sin embargo, dependiendo de las pruebas que se quieran realizar, las herramientas también presentan desventajas. Para las pruebas de funcionalidad, Foundry no realiza *shrinking*, por lo que no minimiza la entrada que da error. En los *tests* de consumo de gas, Echidna no indica la condición por la que ha fallado el `assert`, no tiene en cuenta el coste consumido por `calldata` y no realiza reportes de gas como Foundry.

En conclusión, el uso combinado de estas herramientas permite realizar el análisis de contratos inteligentes desde diferentes perspectivas. Mientras que Foundry resulta más eficaz para el análisis del consumo de gas y la detección de la vulnerabilidad de reentrada, Echidna aporta una mejor capacidad para pruebas funcionales sobre los

contratos.

## 6.2. Líneas de trabajo futuro

Tomando este trabajo como base, se presentan las siguientes líneas de trabajo futuro.

### 6.2.1. Pruebas con anvil

Para las pruebas realizadas con Foundry sólo se ha hecho uso de la herramienta `forge`. La herramienta `anvil` simula un nodo de Ethereum, por lo que se podrían realizar *tests* específicos en los que se necesita desplegar los contratos en la red.

Anvil simula todas las condiciones del mundo real y permite al usuario ejecutar pruebas sin ningún tipo de consecuencia [19].

Sería interesante usar `anvil` para evaluar el comportamiento de los contratos en un entorno más realista.

### 6.2.2. Estudio de comandos de Echidna

Como se ha explicado anteriormente, Echidna tiene varios modos de *testing*. En este trabajo nos hemos centrado en usar los modos `assertion`, para pruebas de funcionalidad y de consumo de gas, y `property` para detectar la vulnerabilidad de reentrada.

Usando el modo de *testing* `exploration` la herramienta generaría secuencias de *fuzzing* pero no centrándose en falsificar las propiedades, sino en maximizar la cobertura de código. En trabajos futuros, este modo puede ser bastante útil para hacer un análisis de cobertura de código en el que se vean partes del código que nunca se ejecutan, y ver cómo se comporta el flujo de ejecución del programa.

Por último, con el modo `overflow` podría utilizarse para ampliar el análisis de vulnerabilidades ya que permite detectar *overflows* o *underflows* en el contrato. Una de las razones por la que un contrato puede dejar de funcionar correctamente es por estos problemas, por lo que detectarlos antes de tiempo puede evitar comportamientos inesperados.

### 6.2.3. Vulnerabilidad de control de acceso

Una vulnerabilidad relevante es la de control de acceso. Esta ocurre cuando las funciones críticas de un contrato no tienen una verificación de permisos adecuada por lo que cualquier persona puede ejecutar la función [13].

Para comprobar que las herramientas son capaces de detectar esta vulnerabilidad se debería de hacer un contrato cuya verificación de permisos en una función no sea adecuada.

Para comprobar si se puede detectar la vulnerabilidad, se puede crear un *test* que invoque dicha función desde distintas direcciones para simular distintos roles de acceso, y comprobar si se ha podido ejecutar la función o no. En Foundry esto se

puede hacer mediante `vm.prank`, que permite modificar temporalmente el valor de `msg.sender`.

## 6.3. Contribuciones personales

Este trabajo se ha realizado en conjunto con mi compañero Mohamed El Farissi, pero por causas sobrevenidas se ha tenido que defender en convocatorias distintas por lo que se añaden aquí mis contribuciones:

Al comienzo del trabajo investigué, junto con Mohamed, los distintos tipos de *testing* aplicables a contratos inteligentes. Una vez encontrada la técnica de *fuzzing*, decidí profundizar en ella, buscando información que pudiera ser útil para el desarrollo de la memoria.

Investigando sobre los tipos de *fuzzing*, encontré la técnica de *mutation-based fuzzing* que mejora el *fuzzing* tradicional porque las entradas generadas no son completamente aleatorias, sino que se basan en otras secuencias generadas que han sido válidas. Profundicé en el estudio de esta variante y redacté un primer borrador sobre el concepto general de generación de entradas con esta técnica. Esta fue la técnica que se decidió estudiar.

Investigué distintas herramientas capaces de realizar *mutation-based fuzzing*. Tras comparar varias alternativas y analizar cuáles se ajustaban mejor a los objetivos del trabajo, decidí centrarme principalmente en Foundry, herramienta compartida con mi compañero, y Echidna.

Instalé Foundry y desarrollé un pequeño contrato con *tests* funcionales simples para familiarizarme con el funcionamiento de la herramienta. Posteriormente diseñé un contrato que sirviera tanto para las pruebas funcionales como para las pruebas de consumo de gas. Este contrato incluía funciones con distintos tipos de complejidad computacional con el objetivo de analizar posteriormente cómo variaba el consumo de gas dependiendo del comportamiento de cada función.

Una vez desarrollado el contrato, implementé las pruebas necesarias para comprobar su correcta funcionalidad. Además, realicé diversos ajustes para evitar desbordamientos en las pruebas que utilizaban bucles `for`. Paralelamente, redacté en Google Documents un primer borrador con los resultados obtenidos durante las pruebas funcionales realizadas con Foundry.

También desarrollé los *tests* de consumo de gas utilizando Foundry. Más adelante, tras la aportación de Mohamed sobre el uso del comando `gas report`, repetí las pruebas ejecutando cada función individualmente para poder analizar con más detalle el consumo de gas de cada una de ellas.

Respecto a Echidna, me encargué de investigar el funcionamiento interno de la herramienta mediante el análisis del código fuente disponible en su repositorio público. Documenté los conceptos clave en un documento compartido y posteriormente profundicé en aspectos internos como el *corpus*, la generación de secuencias y el proceso de *shrinking* utilizado cuando un *test* falla.

Tras instalar Echidna, desarrollé pruebas para el mismo contrato utilizado con Foundry empleando inicialmente el modo de *testing* predeterminado de la herramienta, basado en pruebas de invariantes. Una vez comprobado el funcionamiento

básico, investigué cómo realizar pruebas funcionales independientes para cada función del contrato. Finalmente, concluí que el modo `assertion` era la alternativa más adecuada, por lo que desarrollé y ejecuté las pruebas necesarias utilizando dicho modo.

Para los *tests* de consumo de gas con Echidna implementé pruebas similares a las realizadas con Foundry, analizando posteriormente los resultados obtenidos y redactando un borrador con el análisis y las capturas correspondientes.

Además, diseñé los contratos necesarios para recrear una vulnerabilidad de reentrada en Echidna y desarrollé distintos *tests* para detectar este comportamiento. Tras probar diferentes comprobaciones, conseguí implementar un *test* adecuado que permitía detectar correctamente la vulnerabilidad de reentrada.

Finalmente, también he contribuido a la redacción y revisión general de la memoria, intentando que todos los conceptos y resultados quedaran correctamente explicados y estructurados.

# Introduction

*"Bitcoin will do to banks what email did to the postal industry."*
— Rick Flkvinge

*"The greatest risk is not taking any risk. In a rapidly changing world, the only strategy that is guranteed to fail is not taking risks."*
— Alistair Cockburn

## 6.4. Motivation

The use of *smart contracts* is becoming increasingly widespread, during the fourth quarter of 2025, 8.7 million new smart contracts were deployed only on Ethereum [10]. These contracts can often manage large amounts of money, so their correct functioning is not optional.

When deploying a contract on Ethereum, it becomes immutable, so if there is any bug it is very difficult to modify it. Furthermore, the compiled code of the contract is public, so if a vulnerable contract is deployed on the network, attackers could also exploit them, causing financial losses or preventing the contract from functioning as intended [21].

It is not the first time that a contract with vulnerabilities is deployed and attackers have managed to exploit the flaw, obtaining large amounts of money. For this reason, testing contracts before using them in the real world is crucial.

Given the importance of testing smart contracts, it is of interest to analyze different tools that allow testing smart contracts, analyzing their internal behavior and the capabilities of each tool. This work focuses on two tools: Foundry and Echidna.

## 6.5. Objectives

The main objective of this work is to study the functioning, perform experiments, and compare different testing tools for smart contracts, specifically Foundry and Echidna. To achieve this objective, the following specific objectives are proposed:

- Study the internal functioning of the tools, and in particular the generation

of sequences with mutation-based fuzzing, and the auxiliary mechanisms used by the tools during this process.

- Design simple smart contracts in order to perform functional tests and gas consumption tests on them, and design specific smart contracts that reproduce the reentrancy vulnerability. The contracts used for gas consumption tests will have functions of different computational cost in order to study different cases.

- Develop tests for each tool in order to verify that the designed contracts work correctly. To evaluate the tool's ability to detect errors in contracts, a bug will be introduced and the information provided by the tools will be studied.

- Develop tests that allow measuring the gas consumed by functions of different complexity and study, through a specific test, whether the tools take into account the cost of `calldata` or not.

- Develop specific tests to check whether the tool can or cannot detect the reentrancy vulnerability through the execution of these tests.

- Study the advantages and disadvantages of the tools according to the tested use cases and perform a comparison between them.

## 6.6. Work plan

To achieve the objectives described in the previous section, the work was organized as follows:

1. **Research on testing techniques:** Study different software testing techniques and decide which one will be used in the work.

2. **Research on testing tools:** Investigate tools that allow testing smart contracts and implement mutation-based fuzzing in order to perform the tests.

3. **Installation of tools:** Set up the environment and install the selected tools, and verify that they are correctly installed.

4. **Analysis of the internal functioning of the tools:** Study for each tool the fuzzing cycle, how inputs are generated, and their auxiliary mechanisms.

5. **Development of smart contracts:** Develop different smart contracts for each tool in order to perform tests on them. For functional and gas consumption tests, the contracts will be simple. In contrast, for the reentrancy detection study, the contract must reproduce this vulnerability.

6. **Development of tests:** Develop tests that evaluate that the contract functions work correctly, measure how much gas they consume, and detect the reentrancy vulnerability.

7. **Evaluation of results:** Evaluate the ability of the tools to detect faults in a contract, the ability to evaluate gas consumption of functions, and whether the tool can detect reentrancy. The information provided by each tool when running the tests is also evaluated.

8. **Writing of the report:** Write the report of the work showing the research carried out, the experiments performed, and the comparison between the tools.

## 6.7. Structure of the report

This section shows an outline of the structure of the chapters and sections of the document:

Chapter 1 explains the motivation of the project, its objectives, the work plan followed to carry out the project, and how the document is organized. Chapter 2 briefly describes what a blockchain platform is, presents Ethereum, software testing and testing for smart contracts. It also explains more specific concepts of software testing, specifically the fuzzing, mutation testing, and mutation-based fuzzing techniques. Chapter 3 analyzes Foundry and Echidna. It explains how they work internally and how they generate new sequences with mutation-based fuzzing. Chapter 4 presents the experiments carried out with each tool and evaluates the results obtained. It is divided into three parts:

- Functional evaluation of contracts: The ability of the tools to detect errors in a contract is tested.

- Gas consumption evaluation: The ability of each tool to analyze the gas consumed by contract functions is evaluated. It also tests whether the tools take `calldata` into account or not.

- Reentrancy evaluation: It is tested whether, through the development of a test, the tools are able to detect the reentrancy vulnerability or not.

Chapter 5 compares the tools. First, each of the analyzed tools is described individually and then they are compared with each other. Finally, Chapter 6 summarizes the conclusions of the work and proposes future lines of work.

# Conclusions and Future Work

## 6.8. Conclusions

The main objective of the work has been to study the functioning of Foundry and Echidna through the analysis of their internal structure and the execution of different tests to evaluate the capabilities of the tools.

Their ability to detect errors in contracts that affected their correct functionality has been evaluated, as well as the measurement of the gas consumption of the functions. For this purpose, the `gasleft` function has been used and, in the case of Foundry, a specific command to generate a gas reports on concrete functions. In addition, the ability of the tools to recreate the reentrancy vulnerability has been analysed and, through the manual creation of specific tests, their capability of detection of this vulnerability.

After carrying out the different use cases, it has been observed that each tool presents different advantages. Foundry stands out for its ease of use, for the gas reports that the tool can generate on each function of the contract, and also for detecting the cost associated with `calldata`. In contrast, Echidna stands out for the variety of testing modes it offers and for the shrinking process it performs when a test fails.

Depending on the tests that are intended to be performed, the tools may also present disadvantages. For functional testing, Foundry does not perform shrinking, so it will not minimise the input that causes the error. In gas consumption tests, Echidna does not indicate the condition under which the `assert` failed, does not take into account the cost consumed by `calldata`, and does not generate gas reports like Foundry.

In conclusion, the combined use of these tools allows smart contracts analysis from different perspectives. While Foundry is more effective for gas consumption analysis and detection of the reentrancy vulnerability, Echidna provides better capability for functional testing of contracts.

## 6.9. Future work

Taking this work as a basis, the following future work directions are presented.

### 6.9.1. Testing with anvil

For the tests carried out with Foundry, only the `forge` tool has been used. The `anvil` tool simulates an Ethereum node, so specific tests where the contracts need to be deployed on the network could be performed.

Anvil simulates all real-world conditions and allows the user to execute tests without any consequence [19].

It would be interesting to use `anvil` to evaluate the behaviour of contracts in a more realistic environment.

### 6.9.2. Study of Echidna commands

As explained previously, Echidna has several *testing* modes. In this work we have focused on using the `assertion` mode for functional and gas consumption tests, and the `property` mode for detecting the reentrancy vulnerability.

Using the `exploration` *testing* mode, the tool would generate fuzzing sequences but not focusing on falsifying properties, but on maximising code coverage. In future work, this mode can be quite useful for performing a code coverage analysis in which parts of the code that are never executed can be identified, and the execution flow of the program can be analysed.

Finally, with the `overflow` mode it could be used to extend the vulnerability analysis since it allows detecting overflows or underflows in the contract. One of the reasons why a contract may stop working correctly is due to these problems, so detecting them early can prevent unexpected behaviour.

### 6.9.3. Access control vulnerability

A relevant vulnerability is access control. This occurs when critical functions of a contract do not have proper permission checks, so anyone can execute the function [13].

To verify that the tools are able to detect this vulnerability, a contract should be created whose permission check in a function is not adequate.

To check if this vulnerability can be detected, a test can be created that invokes that function from different addresses to simulate different access roles, and verify whether the function could be executed or not. In Foundry this can be done using `vm.prank`, which allows temporarily modifying the value of `msg.sender`.

## 6.10. Personal Contributions

This work was carried out jointly with my colleague Mohamed El Farissi, but due to some circumstances it had to be defended in different sessions, so my contributions are included here:

At the beginning of the work I investigated, together with Mohamed, the different types of testing applicable to smart contracts. Once the fuzzing technique was found, I decided to go deeper into it, searching for information that could be useful for the

development of the thesis.

While researching fuzzing types, I found the mutation-based fuzzing technique, which improves traditional fuzzing because generated inputs are not completely random, but are based on other previously generated valid sequences. I studied this variant in depth and wrote a first draft about the general concept of input generation using this technique. This was the technique that was finally chosen for study.

I investigated different tools capable of performing mutation-based fuzzing. After comparing several alternatives and analysing which ones best fit with the objectives of the work, I decided to focus mainly on Foundry, a tool shared with my colleague, and Echidna.

I installed Foundry and developed a small contract with simple functional tests to become familiar with the tool. Later, I designed a contract that could be used both for functional and gas consumption testing. This contract included functions with different computational complexity in order to later analyse how gas consumption varied depending on each function's behaviour.

Once the contract was developed, I implemented the necessary tests to verify its correct functionality. In addition, I made several adjustments to avoid overflows in tests that used `for` loops. In parallel, I wrote an initial draft in Google Documents with the results obtained during the functional tests performed with Foundry.

I also developed gas consumption tests using Foundry. Later, after Mohamed's contribution regarding the use of the `gas report` command, I repeated the tests executing each function individually in order to analyse in more detail the gas consumption of each of them.

Regarding Echidna, I was responsible for investigating the internal workings of the tool by analysing the source code available in its public repository. I documented the key concepts in a shared document and later went deeper into internal aspects such as the corpus, sequence generation, and the shrinking process used when a test fails.

After installing Echidna, I developed tests for the same contract used with Foundry, initially using the default testing mode of the tool based on invariant testing. Once the basic behaviour was verified, I investigated how to perform independent functional tests for each contract function. Finally, I concluded that the `assertion` mode was the most appropriate option, so I developed and executed the necessary tests using that mode.

For gas consumption tests with Echidna I implemented tests similar to those performed with Foundry, later analysing the results and writing a draft with the analysis and corresponding figures.

In addition, I designed the necessary contracts to recreate a reentrancy vulnerability in Echidna and developed different tests to detect this behaviour. After trying different checks, I successfully implemented a suitable test that correctly detected the reentrancy vulnerability.

Finally, I also contributed to the writing and general revision of the thesis, ensuring that all concepts and results were correctly explained and structured.

# Bibliografía


[1] Foundry. https://github.com/foundry-rs/foundry, 2025.

[2] Medusa. https://github.com/crytic/medusa, 2025.

[3] Revm. https://github.com/foundry-rs/revm, 2025.

[4] Sumo. https://github.com/MorenaBarboni/SuMo-SOlidity-MUtator, 2025.

[5] Anvil. https://www.getfoundry.sh/anvil, 2026.

[6] Cast. https://www.getfoundry.sh/cast, 2026.

[7] Chisel. https://www.getfoundry.sh/chisel, 2026.

[8] Forge. https://www.getfoundry.sh/forge, 2026.

[9] Elvira Albert, Emanuele De Angelis, Marco Di Ianni, Fabio Fioravanti, and Pablo Gordillo. Towards Property-Based Testing of Smart Contracts Using Gas Analysis. In Massimo Bartoletti and Diego Marmsoler, editors, *7th International Workshop on Formal Methods for Blockchains (FMBC 2026)*, volume 142 of *Open Access Series in Informatics (OASIcs)*, pages 9:1–9:8, Dagstuhl, Germany, 2026. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[10] Ayesha Aziz. Ethereum despliega un récord de 8,7 millones de contratos inteligentes en el cuarto trimestre. https://coinmarketcap.com/academy/es/article/ethereum-deploys-record-87m-smart-contracts-in-q4, 2026.

[11] Morena Barboni, Andrea Morichetta, and Andrea Polini. Sumo: A mutation testing approach and tool for the ethereum blockchain. *Journal of Systems and Software*, 193:111445, 2022.

[12] BBC News. Digital currency ethereum is tanking because of a $50m hack. https://www.bbc.com/news/technology-36585930, 2016.

[13] Fernando Boiero. Seguridad de smart contracts: 10 vulnerabilidades comunes y como prevenirlas. https://www.xcapit.com/es/blog/smart-contract-security-common-vulnerabilities, 2025.

[14] Miriam Martínez Canelo. Qué es el testing de software. https://profile.es/blog/que-es-el-testing-de-software/#Objetivos_del_testing_de_software, s.f.

[15] Chainalysis. Curve finance pools exploited due to code vulnerabilities. https://www.chainalysis.com/blog/curve-finance-liquidity-pool-hack/, 2023.

[16] ConsenSys Diligence. Smart contract best practices: Denial of service (dos). https://consensys.github.io/smart-contract-best-practices/attacks/denial-of-service/.

[17] CyTAL. Mutation-based fuzz testing: A practical guide with protocrawler, s.f.

[18] S.W. Driessen, D. Di Nucci, D.A. Tamburri, and W.J. van den Heuvel. SolAR: Automated test-suite generation for solidity smart contracts. *Science of Computer Programming*, 232:103036, 2024.

[19] Eunice. Foundry series (part 3) - the core components of foundry: Anvil, forge, cast chisel. https://medium.com/@unixfundz/foundry-series-part-3-the-core-components-of-foundry-anvil-forge-cast-chisel-193c7469c586, 2025.

[20] Ethereum Foundation. Introducción técnica a ethereum. https://ethereum.org/es/developers/docs/intro-to-ethereum/, 2026.

[21] Ethereum Foundation. Prueba de contratos inteligentes. https://ethereum.org/es/developers/docs/smart-contracts/testing/, 2026.

[22] GitLab. ¿qué es la prueba fuzzing? https://about.gitlab.com/es/topics/devsecops/what-is-fuzz-testing/, s.f.

[23] ETH Kipu. Cómo funciona la evm. https://edp.ethkipu.org/modulo-1/intro-a-smart-contracts/ethereum-101/evm/como-funciona-la-evm, 2025.

[24] La Cripto Guía. Echidna. https://lacryptoguia.com/glossary/echidna/, 2026.

[25] Bitstamp Learn. ¿qué es la inmutabilidad de la blockchain? https://www.bitstamp.net/es/learn/security/what-is-blockchain-immutability/, 2022.

[26] Leo. Eip-1153: Transient storage. https://medium.com/@sunnybin1007/eip-1153-transient-storage-d22d5b19f6f7, 2025.

[27] Faizan Nehal. Understanding the stack based architecture of evm. https://faizannehal.medium.com/understanding-the-stack-based-architecture-of-evm-af45dc9819f2, 2023.

[28] Paradigma. Mejora la calidad de los tests con mutation testing. https://www.paradigmadigital.com/dev/mejora-calidad-tests-mutation-testing/, 2025.

[29] Chema Polo. Explorando la ethereum virtual machine (evm), bytecode y opcodes: El corazón de los contratos inteligentes en ethereum. https://informaticapolo.com/explorando-la-ethereum-virtual-machine-evm-bytecode-y-opcodes-el-corazon-de-los-contratos-inteligentes-en-ethereum/, 2024.

[30] QAlified. Aprende las diferencias entre pruebas manuales y pruebas automatizadas. https://qalified.com/es/blog/manual-vs-automatizadas-software-pruebas/, 2023.

[31] Régis. Fuzz testing smart contracts with echidna: A practial guide to finding vulnerabilities. https://medium.com/@regis-graptin/fuzz-testing-smart-contracts-with-echidna-a-practical-guide-to-finding-vulnerabilities-e98bb8da1408, 2025.

[32] Banco Santander. Smart contracts, ¿qué son y para qué sirven? https://www.santander.com/es/stories/smart-contracts, 2022.

[33] Securities and Exchange Commission (SEC). Report of investigation pursuant to section 21(a) of the securities exchange act of 1934: The dao. Technical report, SEC, 2017.

[34] Kyle Siegrist. 5.25: The irwin-hall distribution. https://stats.libretexts.org/Bookshelves/Probability_Theory/Probability_Mathematical_Statistics_and_Stochastic_Processes_(Siegrist)/05:_Special_Distributions/5.25:_The_Irwin-Hall_Distribution, 2022.

[35] Trail of Bits and community contributors. Echidna: A fast smart contract fuzzer. https://github.com/crytic/echidna, 2023.

[36] Vyperlang Security Advisory. Vyper has incorrectly allocated named reentrancy locks (ghsa-5824-cm3x-3c38). https://github.com/vyperlang/vyper/security/advisories/GHSA-5824-cm3x-3c38, 2023.

[37] Gavin Wood et al. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper*, 151(2014):1–32, 2014.

[38] Stephanie Susnjara y Ian Smalley. ¿qué es el blockchain? https://www.ibm.com/es-es/think/topics/blockchain, s.f.

Apéndice A

# Código pruebas

El código de las pruebas realizadas se encuentra en el siguiente repositorio de github:

https://github.com/aligrado/TFG-testing-smart-contracts